

Communication Protocols and Drivers

Communication Protocols and Drivers

*From a protocol of your own to a
tested CAN driver in C++*

Erik Pihl

12 September 2026

Communication Protocols and Drivers

Course book. One chapter per lecture.

The book takes you from a protocol of your own, where every mechanism is written by hand, to a driver for a CAN controller in an FPGA that a parallel class is designing at the same time. The last third connects the two halves on real hardware.

This is the English edition of *Kommunikationsprotokoll och drivrutiner*. Both editions are set from the same course material, and the two are kept in step chapter by chapter.

Text and figures: Erik Pihl.

The text and figures of this book, like the course material they are set from, are licensed under the Creative Commons Attribution 4.0 International licence (CC BY 4.0). You may copy, distribute and adapt the material for any purpose, including commercially, provided that you credit the author and the licence. The licence is at <https://creativecommons.org/licenses/by/4.0/>. The code in the

book's listings may also be used under the MIT licence, which is the licence for all code in the course repository. The lecture material and the suggested solutions to the exercises live there too.

The code is C++17. The driver keeps to a subset that can be cross-compiled freestanding for an AVR32DB28, that is, without heap, exceptions and RTTI. AVR, AVR-Dx, Cyclone V and DE0-CV are trademarks of their respective owners.

Set with Lua[®]TeX in TeX Gyre Pagella and DejaVu Sans Mono.

This edition: 12 September 2026.

Contents

Preface	ix
1 Frames	1
1.1 The problem with a byte stream	1
1.2 The fields of the frame	2
Summary	4
Exercises	5
2 Byte parsing	7
2.1 From buffer to stream	7
2.2 Robustness	9
Summary	10
Exercises	11
3 Buses, routing and reliability	13
3.1 The bus and the routing	13
3.2 A deterministic error model.	14
3.3 Reliable delivery	15
Summary	17
Exercises	17
4 CAN, the register map and the architecture	19
4.1 What the hardware takes over	19
4.2 The controller, and why a layer sits in between	20
4.3 The register map	20
4.4 The layered architecture	23
Summary	24
Exercises	25
5 The frame in code, and the first test suite	27
5.1 <code>driver::can::Frame</code>	27
5.2 The test pyramid and the supplied suites	29

	Summary	30
	Exercises	31
6	The interface and the stub	33
6.1	The interface	33
6.2	The stub	34
	Summary	35
	Exercises	36
7	Component testing through the stub	39
7.1	The bug a unit test cannot see.	39
7.2	<code>app::EchoNode</code>	40
7.3	Reading the component test.	41
	Summary	42
	Exercises	43
8	SPI and the transaction protocol	45
8.1	SPI on four wires	45
8.2	The transaction	45
8.3	What it costs in time	47
	Summary	47
	Exercises	48
9	The transport seam and the register access	51
9.1	Why yet another seam	51
9.2	The constants	52
9.3	The two helper functions	53
	Summary	55
	Exercises	56
10	The driver	59
10.1	<code>send()</code>	59
10.2	The packing	60
10.3	<code>receive()</code>	61
10.4	<code>hasError()</code> and <code>clearError()</code>	62
10.5	Waiting for a transmission.	62
10.6	Checklist	63
	Summary	64
	Exercises	64

11	Testing without hardware	67
11.1	Why two	67
11.2	ScriptedTransport	68
11.3	BankTransport	68
11.4	The whole stack on a laptop.	69
11.5	From a failing test case to the bug	70
11.6	What a green suite does not prove	70
	Summary	70
	Exercises	71
12	The hardware, the transport and the factory	73
12.1	volatile	73
12.2	The SPI peripheral	74
12.3	MVIO: two voltages in the same chip	75
12.4	Freestanding C++	76
12.5	The factory	76
12.6	main	77
12.7	An application you can debug with	78
	Summary	78
	Exercises	79
13	Bring-up	81
13.1	The wiring	81
13.2	The ladder	82
13.3	Troubleshooting	83
	Summary	84
	Exercises	84
14	Bus analysis	87
14.1	Why an analyser.	87
14.2	Reading a trace	88
14.3	Looking for the simplifications	88
14.4	Bus load	88
14.5	One frame, all the way	89
	Summary	89
	Exercises	89
15	Revision	91
15.1	The same questions, two answers	91
15.2	What the written test covers.	92
15.3	The most expensive mistakes	92

15.4	After the course	93
A	Solutions	95
	Chapter 1 — Frames	95
	Chapter 2 — Byte parsing	96
	Chapter 3 — Buses, routing and reliability	98
	Chapter 4 — CAN, the register map and the architecture	99
	Chapter 5 — The frame in code	101
	Chapter 6 — The interface and the stub	103
	Chapter 7 — Component testing through the stub	104
	Chapter 8 — SPI and the transaction protocol	106
	Chapter 9 — The transport seam	108
	Chapter 10 — The driver	109
	Chapter 11 — Testing without hardware	111
	Chapter 12 — The hardware and the factory	112
	Chapter 13 — Bring-up	114
	Chapter 14 — Bus analysis	116
B	About the written test	119
C	Practice test	121
D	Answers to the practice test	125

Preface

This book is about two ways of solving the same problem.

The first part solves it in software. You decide for yourself where a frame begins, how long it is, how it is protected against errors, and what should happen when a reply never arrives. Every mechanism becomes your own, and every mechanism becomes code you have to write and maintain. That is instructive in a way that is hard to replace: you understand what a length field is for only once you have forgotten one.

The second part throws all of that away and lets the hardware do the work. CAN dates from 1986, it sits in every car built since the mid-nineties, and it solves very nearly the problems you have just solved yourself — except in silicon, and better. What is left for the software is talking to the controller.

The third part connects it to reality: a microcontroller, an FPGA, four wires between them, and a bus a bus analyser can read.

What makes the course unusual

The CAN controller you write a driver for does not exist yet. It is being designed in VHDL by a parallel class, at the same time as you write the software against it.

Neither class can compile against the other's code. The only thing binding the two halves together is two documents — a register map and an SPI protocol — and they are the project's real requirements specification.

Note: Working against a contract instead of against an implementation is not a classroom exercise. It is how embedded development works when hardware and software are built in parallel, which is the normal case and not the exception. Most of what is hard in this course is hard for that reason.

How the book is arranged

One chapter per lecture, fifteen of them, in three parts:

Chapters	Part
1–3	A protocol of your own. Frames, byte parsing, buses, routing and reliability.
4–12	The CAN driver. The register map, the architecture, the interface, the stub, SPI, the transport seam, the driver, the testing and the hardware.
13–15	The hardware. Bring-up, bus analysis, and revision for the written test.

Every chapter opens with what it contains, ends with a summary, and has its exercises after that. **The solutions to every exercise that does not ask for code are in appendix A** — all the reasoning, calculation and decoding exercises, and the parts of the coding exercises that are reasoning. The code that the coding exercises ask for is not answered there: for that, the supplied test suite is the answer, and the code is published in the course repository after the lecture in question.

The practice exam with full answers is last, in the appendix.

Assumed knowledge

The book assumes classes, inheritance, virtual functions, abstract base classes and the factory pattern. It does not revise them. It also assumes basic familiarity with Git, a terminal and make.

Nothing about CAN, about hardware registers or about testing is assumed.

Further reading

One more book belongs to the course, and it is not needed to pass it: *CAN — bussen, framen och kontrollern*, the course’s own book about CAN as a protocol. It covers the bus, the frame, bit stuffing, CRC-15, arbitration, bit timing and error handling, with exercises and answers, and is to be regarded as the long version of this book’s chapter 4. It lives in a repository of its own, because it is shared with the hardware course and therefore mentions no course by name.

CHAPTER 1

Frames

LECTURE 1 · A PROTOCOL OF YOUR OWN

Why a byte stream is not communication, and what has to be added.

IN THIS CHAPTER

- What a communication protocol is, and which problem it solves.
- Why an unstructured byte stream cannot be interpreted.
- The fields of the frame: SOF, LEN, TYPE, DST, SRC, SEQ, DATA and CHK.
- Serialisation and deserialisation, and endianness as a source of bugs.

1.1 The problem with a byte stream

Two systems are connected by a serial link — UART, RS-485, SPI, anything at all. One sends bytes, the other receives them. That is all the link does.

Suppose the sending node wants to send the temperature 23 degrees and the humidity 45 per cent. It sends two bytes:

```
17 2D
```

The receiver gets two bytes. And now come the questions, one after the other:

- **Where does the message begin?** If the receiver was connected in the middle of a transfer, 17 may well be the second byte of something else.
- **How long is it?** Are more bytes coming, or was that all?
- **What do they mean?** That 17 is the temperature and 2D the humidity is an agreement that exists nowhere in the stream.
- **Did they arrive intact?** A disturbance that changed 17 into 16 looks exactly like a temperature of 22 degrees.
- **Who sent them, and to whom?** On a bus with more than two nodes that is not obvious.

None of the questions can be answered from the bytes themselves. The answers have to be added, and what adds them is a **protocol**.

1.1.1 Frame versus stream

A **frame** is a delimited, self-describing unit in the stream. It says where it begins, how long it is, what it contains, who it comes from, who it is for, and whether it arrived intact.

The stream is still nothing but bytes — it always is. The difference is that the bytes are now arranged according to an agreement both sides know about, and one that can be checked.

Note: This is the only idea in the chapter, and the rest of the book is its consequences. Both the protocol of your own in chapter 2 and CAN in chapter 4 solve the same five questions. They differ in *where* the solution sits: in your code, or in silicon.

1.2 The fields of the frame

The course's protocol uses the following structure. The number is the length of the field in bytes:

SOF (2) | LEN (1) | TYPE (1) | DST (1) | SRC (1) | SEQ (2) | DATA (N) | CHK (2)

Field	Bytes	What it solves
SOF	2	<i>Where does the message begin?</i> A fixed pattern, 0xA5F7, marking the start.
LEN	1	<i>How long is it?</i> The number of data bytes that follow.
TYPE	1	<i>What does it mean?</i> The kind of message.
DST	1	<i>To whom?</i> The receiver's address.
SRC	1	<i>From whom?</i> The sender's address.
SEQ	2	A sequence number, which makes it possible to pair a reply with its request and to recognise a duplicate.
DATA	N	The payload, LEN bytes long.
CHK	2	<i>Did it arrive intact?</i> The sum of all preceding bytes.

The frame types in the course's protocol:

Type	Value
Ping	0x00
Pong	0x01
StatusRequest	0x02
StatusResponse	0x03
Ack	0x04
Nack	0x05

1.2.1 Why SOF is two bytes

One byte is not enough. Whichever value you pick will sooner or later turn up in the middle of a payload, and then the receiver finds a frame that does not exist.

Two bytes make it unlikely rather than impossible — `0xA5F7` can still occur in data. That is why SOF on its own is never sufficient: the receiver also has to check that the length is plausible and that the checksum is correct before it believes the frame. Chapter 2 builds that check.

1.2.2 Why LEN is needed when DATA runs out anyway

Because the receiver reads one byte at a time and cannot look ahead. Without a length field it never knows whether the next byte is data or checksum.

The alternative would be an end marker, but then that pattern has to be impossible in the payload — which requires re-encoding the data. CAN solves the same problem with bit stuffing (see the course's CAN book); here a length field is enough.

1.2.3 The checksum

The course's checksum is simple: a **16-bit sum of every byte up to the CHK field**.

```
std::uint16_t computeChecksum(const std::uint8_t* buf, const std::size_t chkOffset)
{
    std::uint16_t checksum{};
    for (std::size_t i{}; i < chkOffset; ++i)
    {
        checksum += buf[i];
    }
    return checksum;
}
```

It detects single bit errors and most short error bursts. What it does not detect is two bytes swapping places, or one byte increasing by one while another decreases by one — the sum is the same. A CRC does better, and that is what CAN uses.

Note: The checksum says that the frame arrived *intact*. It says nothing about it arriving *at all*. The difference between integrity and delivery guarantee is the whole second half of chapter 3, and it is worth keeping the two apart from the start.

1.2.4 Serialisation and endianness

To serialise is to write the fields of the frame into a buffer in the right order. Fields wider than a byte — SOF, SEQ and CHK — have to be written in a definite byte order, and the course's protocol uses **big-endian**: most significant byte first.

```
SEQ = 0x7F05  -> 7F 05      (big-endian, what the protocol says)
               -> 05 7F      (little-endian, what your x86 does in memory)
```

This is the classic bug in all protocol code, and it is unpleasant for two reasons. It is invisible when both sides have the same bug, and it looks like random corruption when they do not: 0x0005 becomes 0x0500, that is, 1280 instead of 5.

So never write out a multi-byte variable by copying memory. Write it byte by byte, with explicit shifts:

```
// Big-endian: most significant byte first, whatever the host's byte order is.
buf[offset + 0U] = static_cast<std::uint8_t>(value >> 8U);
buf[offset + 1U] = static_cast<std::uint8_t>(value);
```

1.2.5 A complete example

A StatusRequest from address 0x17 to address 0x25, with sequence number 0x7F05 and no payload:

```
A5 F7 00 02 25 17 7F 05 02 5E
|---| | | | |---| |---|
SOF LEN TYPE      SEQ  CHK
      DST--^ ^--SRC
```

The checksum: $A5 + F7 + 00 + 02 + 25 + 17 + 7F + 05 = 606 = 0x025E$.

Note that DST comes before SRC. The order is arbitrary but has to be the same on both sides, and mixing them up gives you a system where every node stubbornly answers itself.

Summary

- A serial link delivers bytes and nothing else. Where a message begins, how long it is, what it means, whether it arrived intact and who it came from are questions the stream does not answer.
- A **frame** is a delimited, self-describing unit that answers them. The course's frame has eight fields: SOF, LEN, TYPE, DST, SRC, SEQ, DATA and CHK.
- **SOF** is two bytes because one byte will sooner or later turn up in the payload. SOF alone is still not enough — the length and the checksum have to be right as well.

- **LEN** is needed because the receiver reads one byte at a time and cannot look ahead.
- **The checksum** gives integrity, not a delivery guarantee. It says that the frame arrived intact, not that it arrived.
- Multi-byte fields are written **big-endian**, byte by byte with explicit shifts — never by copying memory.

Exercises

The solutions to the exercises that do not ask for code are in [appendix A](#). The code is published in the course repository after the lecture.

Exercise 1.1 Build a frame

CALCULATION

A node with address 0x17 sends a `StatusRequest` to a temperature sensor with address 0x25. The sequence number is 0x7F05.

- Write out the whole frame byte by byte, in hexadecimal.
- The sensor answers with a `StatusResponse` containing the temperature 0x16 (one byte). Write out that frame as well.
- How many bytes is the shortest possible frame in this protocol?

Exercise 1.2 The fields and their purpose

REASONING

For each of the fields `SOF`, `LEN`, `TYPE`, `DST`, `SRC`, `SEQ` and `CHK`: describe in one sentence what breaks if the field is removed.

Two of them can be removed without the protocol ceasing to work in a system with exactly two nodes. Which, and why?

Exercise 1.3 Endianness

REASONING

A group serialises the sequence number by copying memory directly:

```
std::memcpy(&buf[Offset::Seq], &frame.seqNr, sizeof(frame.seqNr));
```

- What ends up in the buffer on an x86 machine if `seqNr` is 0x7F05?
- The code nevertheless works flawlessly in their own tests. Why?
- When does it break?

Exercise 1.4 The limits of the checksum

REASONING

The course's checksum is a sum of bytes.

- a) A disturbance swaps two data bytes. Is it detected?
- b) A disturbance increases one byte by 1 and decreases another by 1. Is it detected?
- c) Give an example of a change to the frame above that the checksum does *not* detect.

CHAPTER 2

Byte parsing

LECTURE 2 · A PROTOCOL OF YOUR OWN

Reality: data arrives one byte at a time, and sometimes the wrong bytes arrive.

IN THIS CHAPTER

- Why a parser is needed once the frame no longer sits ready in a buffer.
- The parser as a state machine, state by state.
- Recovery: junk before SOF, dropped bytes, a broken checksum.
- Where validation belongs, and why not in the state machine.

2.1 From buffer to stream

In chapter 1 we assumed that a complete frame was sitting in a buffer. It never is.

A serial link delivers one byte at a time, whenever it arrives, and a node waiting for the next byte must not stop and wait — it has other things to do. Reception therefore has to be **incremental**: take one byte, do whatever can be done with it, and return.

That is exactly what a state machine is for.

2.1.1 The states

One state per field, plus one meaning done:

```
WaitForSof1 -> WaitForSof2 -> WaitForLen  -> WaitForType -> WaitForDst
              -> WaitForSrc  -> WaitForSeq1 -> WaitForSeq2 -> WaitForPayload
              -> WaitForChk1 -> WaitForChk2 -> Ready
```

The parser's public interface is small:

```
class Parser final
{
public:
```

```

    /** Feed the parser one byte. Returns false on a parse error. */
    bool processByte(std::uint8_t byte) noexcept;

    /** True when a complete frame has been framed. */
    bool isFrameReady() const noexcept;

    /** Validate and copy out the framed data. */
    bool extractFrame(Frame& frame) const noexcept;

    /** Discard any partial frame and start over. */
    void reset() noexcept;
};

```

2.1.2 Two responsibilities, deliberately kept apart

Note that `isFrameReady()` and `extractFrame()` are two different things, and that this is deliberate:

- `isFrameReady()` says that the **framing** is done — the right number of bytes have arrived, in the right places.
- `extractFrame()` says that the frame is **valid** — SOF is right, the length is plausible, the type exists, and the checksum adds up.

A frame can be fully framed and still be broken. That is the normal case when something has gone wrong on the wire, and that is why the two questions have to be asked separately.

```

bool Parser::extractFrame(Frame& frame) const noexcept
{
    if (!isFrameReady()) { return false; }

    // Deserialize validates SOF, payload length, frame type and checksum, so a
    // frame that was
    // framed correctly but arrived corrupted is rejected here.
    return frame.deserialize(myBuf, static_cast<std::size_t>(myParsedBytes));
}

```

The validation therefore lives in `Frame::deserialize()` and not in the state machine. That is no accident: the state machine knows where the fields *are*, the frame knows what they *mean*, and mixing the two responsibilities gives you a parser that has to change every time the protocol does.

Note: `extractFrame()` is `const` and therefore cannot reset the parser. That is a deliberate choice: the caller knows whether it wants to try again or throw the frame away, and calls `reset()` itself. The same pattern returns in the driver: a method reports, the caller decides.

2.2 Robustness

A parser that only works when everything goes right is not a parser. Three cases have to be handled, and all three happen for real.

2.2.1 Junk before SOF

The node is connected in the middle of an ongoing transfer, or a disturbance adds bytes. The stream looks like this:

```
32 74 00 FF A5 F7 03 03 25 17 7F 05 10 20 30 02 C2
|-----| |-----|
      junk           the frame
```

The solution falls out of the state machine for free: in `WaitForSof1`, every byte that is not `0xA5` is discarded. The parser stands still until the stream starts to look like a frame.

One case is worth thinking through in particular: the parser is in `WaitForSof2` and gets a byte that is not `0xF7`. The naive answer is to go back to `WaitForSof1` and discard the byte. But if the byte was `0xA5`, you have just discarded the start of a real frame. The right answer is to stay in `WaitForSof2`: the byte you just saw may be the first of the new frame.

2.2.2 Dropped bytes

A byte disappears. Every following field is then shifted by one step, the length field reads a data byte as the length, and the parser waits for the wrong number of bytes.

There is no elegant rescue, and there should not be one: the frame *is* lost. The point is that the parser should detect it and get back to a known state, not that it should guess. The checksum fails, `extractFrame()` returns `false`, the caller calls `reset()`, and the next `0xA5F7` in the stream starts over.

2.2.3 An implausible length

This is the case that is easy to miss, and the only one in the chapter that is a security bug rather than mere error handling.

The length field is one byte and can therefore be up to 255. The parser's buffer holds `MaxPayloadLen` bytes. If the parser receives a length greater than that, and then obediently writes in that many data bytes, it writes past the end of its own buffer.

```

case State::WaitForLen:
{
    // Reject a length the buffer cannot hold, else the payload would overflow ↪
    ↪ myBuf.
    if (MaxPayloadLen < byte) { return reportParseError(); }

    myBuf[myParsedBytes++] = byte;
    myState                = State::WaitForType;
    break;
}

```

Rule: Every length that comes from outside is to be checked against the size of the buffer *before* it is used, not after. The rule holds everywhere in this book, and it holds for TX_DLC in chapter 10 for exactly the same reason.

2.2.4 Two frames in a row

When a frame is complete and extracted, the parser has to be reset before the next one can be received. Forget that, and it stays in Ready and discards every new byte.

That is why `reset()` is in the public interface, and that is why the node in chapter 3 calls it after every extracted frame.

Summary

- Data arrives one byte at a time. Reception therefore has to be incremental, and a state machine with one state per field is the natural way to do it.
- **Framing and validation are two questions.** `isFrameReady()` says that the right number of bytes have arrived; `extractFrame()` says that they mean something. A fully framed frame may very well be broken.
- The validation belongs in `Frame::deserialize()`, not in the state machine: the machine knows where the fields are, the frame knows what they mean.
- Junk before SOF is handled by `WaitForSof1` discarding everything that is not `0xA5`. In `WaitForSof2` the parser is to **stay put** if the byte is `0xA5` again.
- A length field from outside has to be checked against the size of the buffer before it is used. Otherwise it is a buffer overflow, not error handling.
- The parser has to be reset between frames.

Exercises

The solutions to the exercises that do not ask for code are in [appendix A](#). The code is published in the course repository after the lecture.

Exercise 2.1 Find the frame

DECODING

A node receives the following byte stream:

```
32 74 00 FF A5 F7 03 03 25 17 7F 05 10 20 30 02 C2
```

- At which index does the frame begin?
- Which bytes belong to it, from SOF up to and including CHK?
- What is the parser to do with the bytes before it?
- What type does the frame have, and which addresses does it travel between?

Exercise 2.2 The SOF trap

REASONING

The parser is in `WaitForSof2` and receives `0xA5`.

- What does the naive implementation do, and what is lost?
- What should it do instead?
- Construct a byte stream where the difference decides whether a valid frame is found or not.

Exercise 2.3 A dropped byte

DECODING

The same frame as in exercise 2.1, but the byte `0x20` is dropped on the way.

- Which field does the parser read as the checksum?
- Will `isFrameReady()` become true?
- What does `extractFrame()` return, and why?
- Which state is the parser in afterwards, and what does it take for it to find the next frame?

Exercise 2.4 The length field

REASONING

A parser does not check the length field against the size of its buffer.

- Construct a byte stream that makes it write past the end of its buffer.
- Why is it not enough to check the length in `extractFrame()`?
- Which general rule follows from that, and where in chapter 10 does the same rule apply again?

Buses, routing and reliability

LECTURE 3 · A PROTOCOL OF YOUR OWN

Several nodes on one shared wire, and everything that is needed when it is not perfect.

IN THIS CHAPTER

- The shared bus, and why broadcast is the norm in embedded systems.
- Routing on DST and SRC, and why it sits in the node and not in the bus.
- An error model that is *deterministic*, and why that is the whole point.
- ACK and NACK, timeout, retransmission — and the duplicates that follow.

3.1 The bus and the routing

Two nodes can be wired directly to each other. Five cannot — that would be ten connections, and the number grows quadratically. The solution is a **shared bus**: every node on the same wire.

That has an immediate consequence: **everyone hears everything**. A bus is broadcast, and that is not something you can opt out of. If you want a message to reach a particular node, you have to write it in the message.

3.1.1 Where the routing sits

The bus's only job is to deliver every byte to every node. It does not read addresses, it knows nothing about frames, it does not even know that a protocol exists.

The routing sits in the node. Every node receives every byte, feeds it to its parser, and once a complete frame has been extracted compares the frame's DST with its own address:

```
void Node::tick() noexcept
{
    if (!myParser.isFrameReady()) { return; }
```

```

Frame frame{};
if (!myParser.extractFrame(frame)) { myParser.reset(); return; }
myParser.reset();

if (frame.dstAddr != myAddr) { return; } // not for us
handleFrame(frame);
}

```

Putting the routing in the node instead of in the bus is not a simplification made for the course's sake. It is how a real bus works: a wire cannot read addresses.

3.1.2 DST and SRC

DST says who the frame is for. SRC says who it came from, and is needed so that the receiver can answer — without it, it does not know where the answer should go.

SEQ pairs a reply with its request. A node that has sent three requests and gets one reply has to be able to tell which of them it answers.

Note: CAN does all of this completely differently: there is no destination address at all, and the identifier describes the *content* of the message instead of its receiver. The comparison is made in chapter 4, and it is one of the most instructive in the course.

3.2 A deterministic error model

A bus that always delivers every byte in the right order is not a bus, but a function call with extra steps. Real buses drop bytes, flip bits and delay deliveries.

The problem with testing error handling is that real errors are random. A test that fails sometimes says nothing, and an error that happened once cannot be reproduced.

The solution is to make the errors deterministic. The bus implementation in the course's simulation counts bytes and does exactly the same thing on every run:

```

// Deterministic fault injection: byte five is dropped, every run, every time.
if (5U == ++myByteCounter) { return; }

```

Three errors are enough to cover most of it:

Error	What happens
Dropped byte	Every following field is shifted. The parser reads the wrong length, waits for the wrong number of bytes, and finally fails on the checksum.
Flipped bit	The frame is structurally correct — right length, fields in the right places — but the checksum does not add up. That is precisely the case the checksum exists for.
Delay	The frame arrives, whole and correct, but later than expected. Nothing breaks, and that is the point: a correct parser does not care <i>when</i> the bytes arrive.

The third case is worth pausing at. A parser that works when the bytes arrive close together but breaks when they arrive far apart has a hidden timing dependency, and that is a bug which never shows itself until the system is moved to slower hardware.

3.3 Reliable delivery

The checksum gives **integrity**: whoever receives a frame knows that it is intact. It gives no **delivery guarantee**: the sender still does not know whether anything arrived.

That difference is this whole section, and the four mechanisms that follow build on each other in turn.

3.3.1 ACK and NACK

The receiver answers. If the frame arrived intact it sends an Ack; if it was broken in a way that can still be answered, it sends a Nack.

The rules for the acknowledgement frame:

- DST and SRC swap places compared with the acknowledged frame.
- SEQ is the **same** as in the acknowledged frame — that is how the sender knows what is being acknowledged.
- LEN is zero.

That SEQ comes along is not a detail. Without it, a sender with two frames in flight does not know which of them arrived.

Note: A Nack can only be sent when the frame is broken in a way that leaves the sender and the sequence number readable. If the checksum is wrong, *no* field can be trusted — including SRC — so in practice a missing acknowledgement is the usual way an error shows itself.

3.3.2 Timeout

And with that: what does the sender do when nothing comes back?

It cannot wait forever, and it cannot tell the difference between a lost frame, a lost acknowledgement and a slow receiver. The only way out is to make up its mind after a certain time: a **timeout**.

Choosing the timeout value is a trade-off with no right answer:

Too short	Too long
Unnecessary retransmissions	Slow error detection
More duplicates	The bus stands still and waits
The bus is loaded more when it is already loaded	The system feels hung

3.3.3 Retransmission

On a timeout the frame is sent again, with the **same** SEQ. That is crucial: a new sequence number would make the retransmission a new message, and the receiver would not be able to recognise it as a repeat.

The number of attempts has to be limited. A sender that tries for all eternity against a node that is switched off does nothing else.

3.3.4 Duplicates

And here the circle closes. Retransmission is exactly what creates duplicates.

The dangerous case is not that the frame is lost — then nothing happened — but that the **acknowledgement** is lost:

A -> B	StatusRequest, SEQ 0x0007	B receives and processes it
B -> A	Ack, SEQ 0x0007	the acknowledgement is lost
A	timeout	
A -> B	StatusRequest, SEQ 0x0007	retransmission
B		...processes it a second time?

From B's point of view these are two identical frames. If it processes both, the command has been carried out twice, and if the command is "increase the setpoint by one" the result is wrong.

The solution is to remember. Every node stores the most recently processed SEQ per sender. If the same SEQ turns up again:

- Do **not** run the application logic again.
- Send the acknowledgement again **anyway**.

The second point is the one that gets forgotten. The sender is still waiting for its acknowledgement, and silence only leads to yet another retransmission.

Note: Four mechanisms, in a chain where each solves the problem the previous one created: the checksum detects errors, the ACK confirms delivery, the timeout detects a missing ACK,

retransmission fixes it — and duplicate detection cleans up after the retransmission. None of them can be left out, and that is why the chain is worth knowing by heart.

3.3.5 And then the hardware does it for us

Everything in this section is code you write, test and maintain. From chapter 4 onwards most of it sits in silicon: CAN does framing, checksumming and collision handling in hardware, and acknowledges in a single bit inside the frame as well.

That does not make this chapter less important. It makes it the background against which CAN can be understood — and a part of what the written test examines.

Summary

- A shared bus is broadcast. Every node hears everything, and **the routing sits in the node**, not in the bus — a wire cannot read addresses.
- DST says who the frame is for, SRC who it came from so that replies are possible, and SEQ pairs a reply with its request.
- An error model is to be **deterministic**. An error that happens on the same byte every run can be tested; a random one cannot.
- The checksum gives **integrity**, not a **delivery guarantee**.
- The chain: the checksum detects errors → the ACK confirms delivery → the timeout detects a missing ACK → retransmission fixes it → duplicate detection cleans up after the retransmission.
- A retransmission has the **same** SEQ. A new one would be a new message.
- A detected duplicate is **not** to run the application logic again, but **is** to be acknowledged again. Silence only gives you yet another retransmission.

Exercises

The solutions to the exercises that do not ask for code are in [appendix A](#). The code is published in the course repository after the lecture.

Exercise 3.1 Routing

REASONING

- Does the routing happen in the bus or in the node? Justify your answer with what a physical wire can and cannot do.
- A node receives a frame with a correct checksum, but DST is not its address. What should it do?
- Why should it *not* send a Nack in that situation?

Exercise 3.2 The error model

REASONING

For each of the errors — dropped byte, flipped bit, delay — answer:

- a) Does the frame end up structurally correct?
- b) Which mechanism detects the error?
- c) Which state is the parser in afterwards?

One of the three errors should cause no problem at all. Which one, and what does it mean if it does anyway?

Exercise 3.3 A lost acknowledgement

DECODING

Node A sends a StatusRequest with SEQ 0x0007 to node B. B receives it, processes it, and sends an Ack. The acknowledgement is lost.

- a) What does A do next, and after what?
- b) What does B see then?
- c) What should B do, and what should B not do?
- d) What would have happened if B had stayed silent instead?

Exercise 3.4 The timeout value

REASONING

- a) What happens if the timeout is too short? Give two consequences.
- b) What happens if it is too long?
- c) What information would be needed to choose the value correctly, and why does a node rarely have it?

Exercise 3.5 The chain

REASONING

List the five mechanisms in the reliability chain, in order. For each one: which problem does it solve, and which new problem does it create?

CAN, the register map and the architecture

LECTURE 4 · THE CAN DRIVER

The hardware takes over, and the project begins.

IN THIS CHAPTER

- Which mechanisms from chapter 3 CAN performs in silicon, and where.
- The controller you write a driver for, and why a register layer sits in between.
- The register map: what every register means and how every status bit is cleared.
- The layered architecture, and who is allowed to know what.

4.1 What the hardware takes over

CAN solves the same problems as chapter 1–chapter 3, in silicon:

Mechanism	Who does it in CAN
Frame synchronisation	The controller: the SOF bit, and seven recessive bits as an end marker.
Length field	The controller: the DLC, four bits, 0–8 data bytes.
Checksum	The controller: CRC-15, computed and checked by the same engine.
Collision handling	The bus itself, through arbitration — without a bus master and without lost time.
Acknowledgement	The controller: a single bit inside the frame, which every receiver pulls low.
Addressing	Does not exist. The identifier describes the content, and every node filters for itself.
Sequence number	Does not exist. If one is needed, the application has to put it in the payload.
Timeout and retransmission	A complete controller does it in hardware. <i>This one does not</i> — see §4.3.4.

What is left for the software is therefore to hand over an identifier, a length and a few data bytes — and to read off how it went.

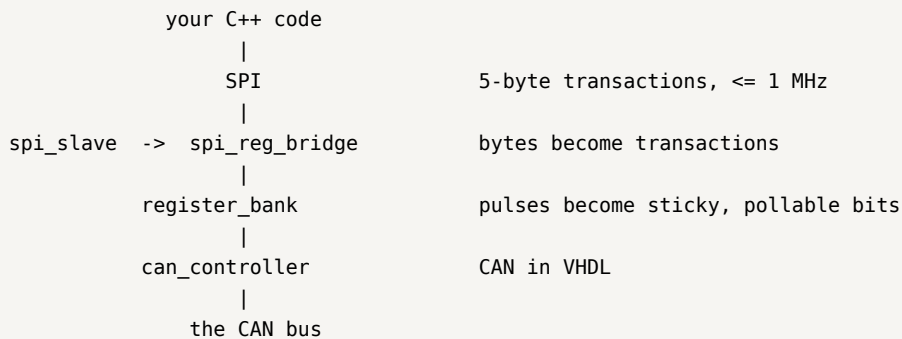
Note: CAN as a protocol — the bus, the frame, bit stuffing, CRC-15, arbitration, bit timing and error handling — is covered in full in the course’s CAN book. This chapter assumes its chapters 1, 2 and 7. Read those first; the rest of that book is further reading you can take when you need it.

4.2 The controller, and why a layer sits in between

The controller is designed in VHDL by a parallel class. It is built for a caller in the same clock domain, and therefore signals with pulses that are **one clock cycle** long — 20 ns at 50 MHz.

A processor polling over a serial bus sees the system a few tens of microseconds at a time at best. A 20 ns pulse is therefore gone thousands of cycles before the next poll arrives.

That is why there are two layers between the controller and your code:



The register bank is the layer that makes the design usable from software. It catches the pulses and holds them as levels that you have to clear yourself. Every “write 0x1 here to acknowledge” in the register map is that mechanism.

4.3 The register map

Thirteen registers, each of them 32 bits. **The index** is what goes out on the wire; the offset column is always $\text{index} \times 4$ and is a legacy of an older, memory-mapped variant of the same design.

Idx	Register	Offset	Acc.	Description
0	STATUS	0x00	R	Bit 0: TX ready. Bit 1: RX valid. Bit 2: Error.
1	TX_ID	0x04	R/W	11-bit transmit ID (bits 10–0).
2	TX_DLC	0x08	R/W	Data Length Code (bits 3–0, value 0–8).
3	TX_DATA_LO	0x0C	R/W	Transmit data bytes 0–3 (byte 0 = MSB).
4	TX_DATA_HI	0x10	R/W	Transmit data bytes 4–7.
5	TX_SEND	0x14	W	Write 0x1 to trigger a transmission.
6	RX_ID	0x18	R	Received identifier.
7	RX_DLC	0x1C	R	Received DLC.
8	RX_DATA_LO	0x20	R	Received data bytes 0–3.
9	RX_DATA_HI	0x24	R	Received data bytes 4–7.
10	RX_ACK	0x28	W	Write 0x1 to acknowledge.
11	ERROR_FLAGS	0x2C	R/W	The error register; write 0x0 to clear.
12	TX_ABORT	0x30	W	Write 0x1 to force TX ready back.

Indices 13–15 are reserved: a read gives 0x00000000, a write is ignored.

4.3.1 STATUS: the three bits everything revolves around

Bit	Meaning	Set by	Cleared by
0	TX ready	reset; a completed transmission; a rising edge on the error level while the bit is low; a write to TX_ABORT	an accepted write to TX_SEND
1	RX valid	a received frame, which latches RX_* at the same time	a write of 0x1 to RX_ACK
2	Error	a rising edge on the controller's error level	a write of 0x0 to ERROR_FLAGS

Bits 31–3 always read as 0.

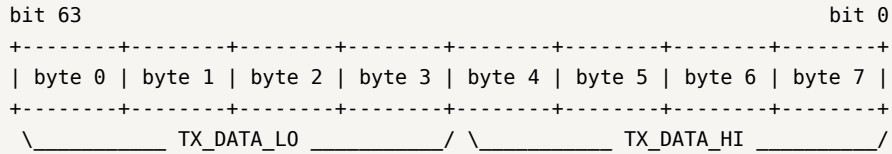
Read that table once more. Every bit has an explicit clear, and that clearing is the driver's responsibility. Forget RX_ACK and bit 1 stays set forever and you read the same frame in an endless loop. Forget that TX_SEND clears bit 0 and you will wonder why the second transmission is never accepted.

4.3.2 Rules that are easy to miss

- TX_SEND and RX_ACK require **bit 0 to be set**; the other bits are ignored.
- ERROR_FLAGS is cleared **only by an all-zero word**. 0x1 does nothing, and 0x000000FF does nothing even though bit 0 is set. The opposite rule to the two above.
- TX_ID masks to eleven bits, TX_DLC to four. A write of 0xFFFFFFFF reads back as 0x000007FF and 0x0000000F respectively.
- Writes to STATUS and RX_* are silently ignored.
- Reads of TX_SEND and RX_ACK give 0x00000000.

4.3.3 Data byte order

TX_DATA and RX_DATA are 64 bits wide, with **data byte 0 in the most significant bits**:



LO and HI refer to the order of the register pair, not to bit significance: **TX_DATA_LO carries the first four data bytes**. That is the most common misreading in the project, and it does not show until a frame arrives back to front.

A frame with dlc = 2 and data AA BB is therefore packed as TX_DATA_LO = 0xAABB0000 and TX_DATA_HI = 0x00000000.

4.3.4 The three limitations

The hardware is simplified compared with real CAN in many respects. Three of them reach all the way up into your code, and have to be handled rather than hidden.

A lost arbitration looks like a successful transmission. STATUS bit 0 comes back in both cases. The bit means *the transmit port is free again*, not *the frame arrived*. If you want to know the outcome: wait for bit 0, and **then** read ERROR_FLAGS.

ERROR_FLAGS is one bit for four causes — lost arbitration, stuff error, failed CRC and a received remote frame. You can see *that* something went wrong, never *what*. Retry logic that tells them apart cannot be written against this interface.

No buffering on the receive side. The controller holds exactly one frame. If the next one arrives before the previous has been acknowledged it is overwritten, and the newest wins. Poll often — and know that at some traffic intensity that will still not be enough.

Note: The hardware validates nothing. A TX_DLC of 0xF is stored and passed on exactly as it is. Rejecting invalid frames is **the driver's** responsibility, and only the driver's: two copies of the same rule are two things that have to be kept in step.

4.4 The layered architecture

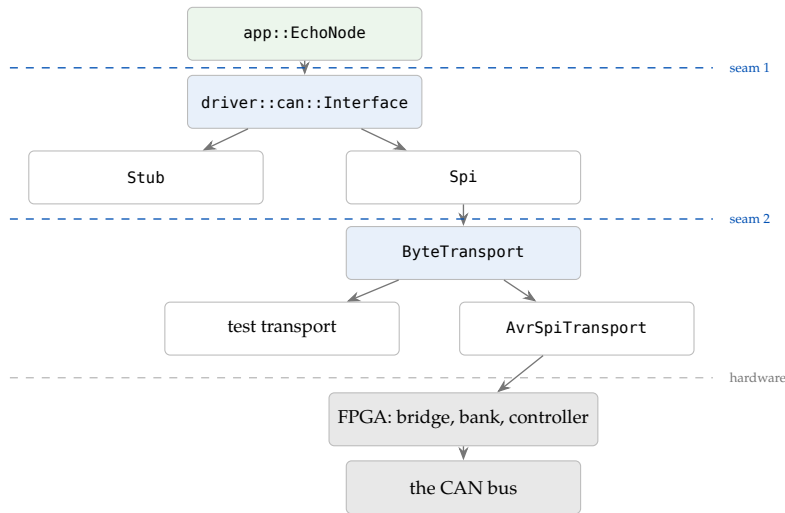


Figure 4.1 The driver stack. Every horizontal line is a boundary somebody is not allowed to look through.

Two seams carry the whole design, and they solve different problems:

Seam	Makes what testable
<code>driver::can::Interface</code>	The application. <code>EchoNode</code> can be run against a <code>Stub</code> and never has to know that hardware exists.
<code>driver::can::ByteTransport</code>	The driver itself. The same <code>Spi</code> code that is flashed to the target can be run on your laptop against a transport you script.

The second seam is not obvious, and it arrives in chapter 9. Without it, `Spi` can only be run on real hardware — and that does not exist until chapter 13, second to last in the course, on a board somebody else is debugging at the same time.

The rule that follows, and the one a reviewer should look for: no file above `driver/can/interface.hpp` may contain the word `SPI`.

4.4.1 The seam has already paid off once

The argument for an interface is usually hypothetical — **what if the hardware is replaced**. Here it is not hypothetical.

An earlier version of this design reached the registers through **memory-mapped I/O** at the base address `0xFF200000`. That assumes a processor with an address bus out to the FPGA fabric, and the microcontroller in this system has no such bus. The registers are reached over SPI instead.

Everything above Interface would have survived that change **unchanged**. That is the whole argument, demonstrated on real code.

4.4.2 The transmit sequence

Sending a frame (id 0x123, dlc 2, data AA BB) is five register writes and one poll:

```

1. read  STATUS          -> bit 0 set? if not: busy, return false
2. write TX_ID           = 0x00000123
3. write TX_DLC          = 0x00000002
4. write TX_DATA_LO      = 0xAABB0000
5. write TX_DATA_HI      = 0x00000000
6. write TX_SEND         = 0x00000001    -> STATUS bit 0 goes low
   ...
7. poll  STATUS          -> bit 0 back: the port is free
8. read  ERROR_FLAGS     -> 0 = the frame went out, 1 = something went wrong

```

And receiving one:

```

1. read  STATUS          -> bit 1 set? if not: no frame, return false
2. read  RX_ID, RX_DLC
3. read  RX_DATA_LO, RX_DATA_HI, unpack, mask against RX_DLC
4. write RX_ACK          = 0x00000001    -> STATUS bit 1 goes low

```

TX_SEND **must** be written last. That write does not mean “store a value” but “start transmitting now”, and the controller then reads the other registers. Write the trigger first and you send the previous frame’s contents — which at bring-up looks like a node running exactly one frame behind.

Summary

- CAN does framing, length, checksum, collision handling and acknowledgement in silicon. What is left for the software is the identifier, the length, the data bytes — and reading off how it went.
- CAN has **no destination address**. The identifier describes the content, and every node filters for itself.
- The controller signals with single-cycle pulses. **The register bank** turns them into sticky bits with an **explicit clear** — and that clearing is the driver’s responsibility.
- TX_SEND and RX_ACK require bit 0 to be set; ERROR_FLAGS is cleared only by an *all-zero* word. Opposite rules, and a common bug.
- TX_DATA_LO carries data bytes **0–3**, with byte 0 in bits 31–24.
- Three limitations reach up into the code: a lost arbitration looks like a successful transmission, the error flag is one bit for four causes, and the receive side does not buffer.

- The hardware validates nothing. The validation lives in the driver, in exactly one place.
- Two seams: Interface makes the application testable, ByteTransport makes the driver testable.

Exercises

The exercises are done with pen and paper, and the solutions are in [appendix A](#).

Exercise 4.1 Who does what

REASONING

Fill in, for every mechanism from chapter 3: who performs it in a CAN system, and where? Frame synchronisation, length field, addressing, sequence number, checksum, acknowledgement, collision handling, timeout and retransmission, duplicate detection.

Two of the rows have no obvious answer. Which, and why?

Exercise 4.2 Pack a frame

CALCULATION

You are to send a frame with `id = 0x2A7`, `dLc = 5` and the data bytes `01 02 03 04 05`. Write out the five register values to be written, in the order they are to be written, in hexadecimal. Why does the order matter?

Exercise 4.3 Unpack a frame

CALCULATION

You read `RX_ID = 0x00000481`, `RX_DLC = 0x00000003`, `RX_DATA_L0 = 0xDEADBEE0` and `RX_DATA_HI = 0x00000000`.

- Which identifier did the frame come from?
- Which data bytes does it contain, in order?
- What are `frame.data[3]` to `frame.data[7]` to contain after `receive()`?
- Which register has to be written before the next frame can be received, and with which value?

Exercise 4.4 Four statements

REASONING

Three of the following describe a bug. Which, and what goes wrong?

- "After `send()` we poll `STATUS` bit 0 and return `true` when it comes back — then we know that the frame arrived."
- "`clearError()` writes `0x1` to `ERROR_FLAGS` to clear the error bit."
- "`receive()` always copies all eight bytes from `RX_DATA_L0/HI` into `frame.data`."
- "`send()` checks `STATUS` bit 0 before it writes `TX_SEND`."

Exercise 4.5 The layer boundary

REASONING

True or false, with a one-sentence justification:

- a) `app::EchoNode` may include `driver/can/spi.hpp`.
- b) `driver::can::Spi` may include `driver/can/interface.hpp`.
- c) `driver::can::Stub` needs a `ByteTransport`.
- d) `ByteTransport` needs to know the register map.
- e) A test of `EchoNode` needs a `ByteTransport`.

CHAPTER 5

The frame in code, and the first test suite

LECTURE 5 · THE CAN DRIVER

Eleven bytes of data, and the tests that decide whether the rest of the course can be trusted.

IN THIS CHAPTER

- `driver::can::Frame`: an aggregate, not an object, and why.
- Where validation belongs when the hardware validates nothing.
- The test pyramid, and why testing is decisive in this course in particular.
- *Reading* a test suite — you write none.

5.1 `driver::can::Frame`

The register map in chapter 4 says exactly what the hardware can send and receive, and it is not much: an identifier, a length and up to eight data bytes. The class mirrors that:

```
namespace driver::can
{
    /**
     * @brief A single CAN frame: identifier, data length code and payload.
     */
    struct Frame
    {
        /** Maximum number of payload bytes in a CAN frame. */
        static constexpr std::uint8_t MaxDataLen{8U};

        std::uint16_t id{};                /**< CAN identifier, 11 bits (0x000 - 0x7FF)↵
        ↵ . */
```

```

    std::uint8_t dlc{};           /**< Data length code, 0 - 8. */
    std::uint8_t data[MaxDataLen]{}; /**< Payload. */
};

/**
 * @brief Check whether a frame is valid, i.e. whether the hardware can send it.
 */
[[nodiscard]] constexpr bool isValid(const Frame& frame) noexcept
{
    return (frame.id <= 0x7FFU) && (frame.dlc <= Frame::MaxDataLen);
}

} // namespace driver::can

```

That is the whole file. No .cpp.

5.1.1 Why an aggregate

Frame deliberately has no constructor, destructor or member function. The reasons, in turn:

It is data, not behaviour. A frame is something that gets moved: from the application to the driver, from the driver into four registers, from the registers out onto the bus.

Aggregate initialisation is readable. With the aggregate, `Frame frame{0x123U, 2U, {0xAAU, 0xBBU}}` works. Add a single user-declared constructor and that line stops compiling. The tests you meet from the next chapter onwards are full of such lines.

Default member initialisation makes `Frame f{}` safe. The `{}` after every member means that a default-constructed frame has zeroed fields. Without them, `receive()` could have handed back junk in the bytes it did not fill.

5.1.2 static constexpr, not #define

`MaxDataLen` has a **type**, a **scope** and exists in the **debugger**. A `#define` has none of that, and on top of that applies from the line it stands on to the end of the file, straight through every header included afterwards.

It is also used as the array size, which guarantees that the size of the array and the bounds check in `isValid()` are the same number.

5.1.3 Why `isValid()` is a free function

Two reasons. It keeps the aggregate intact — a member function is the first step towards a class with invariants, and the next step is a constructor. And it belongs to the *driver*, not to the data: the rule comes from this hardware’s register widths, not from CAN as a concept.

Rule: The hardware validates nothing. The driver **must** therefore validate — and **only** the driver. The temptation to put one more check in the application “just to be safe” is to be resisted: two copies of the same rule are two things that have to be kept in step, and the day `MaxDataLen` changes, only one of them will change with it.

5.2 The test pyramid and the supplied suites

5.2.1 You write no tests

Every test suite in this course is **supplied**. Your job is to run them, read them when they fail, and get them to pass.

That is a deliberate arrangement. Writing good tests is a craft of its own, and it comes in a later course. Here you get to see what tests are *for* instead, on a project where they do real work — and somebody who has seen a test suite find a bug in their own code has a far more concrete picture of what a good test is than somebody asked to write one first.

5.2.2 Why testing decides this course

In earlier projects you could run the program and look. Here you cannot:

- **The hardware does not exist yet.** It is being built in parallel. The first time you can run against it is chapter 13, the third session from the end.
- **The hardware is not yours.** When it does exist, somebody else is debugging it at the same time. A bug can be in your code, in their VHDL, or in a jumper wire.
- **Most of it can be run without hardware.** That is not a consolation prize but a design consequence: the seams in chapter 4 exist precisely for it.

The rule: everything that can be run on the laptop is to be green before it is run on hardware. Bring-up is for hunting wiring errors, not bit-packing errors.

5.2.3 The three levels of the pyramid

Level	Where	What
Unit test	chapter 5, chapter 6	One class at a time: <code>Frame</code> , <code>Stub</code> .
Component test	chapter 7, chapter 11	Several parts together, without hardware: <code>EchoNode</code> through the stub; <code>Spi</code> against a simulated register bank.
Integration test	chapter 13, chapter 14	Real hardware, real wires.

The width is a recommendation about *numbers*: many fast tests at the bottom, few expensive ones at the top. A project with three fast tests and one person re-plugging wires is the pyramid upside down.

5.2.4 Reading a test case

Every supplied test case has three parts:

```
TEST(Frame, isValidRejectsTooLargeDlc)
{
    // Arrange.
    driver::can::Frame frame{};
    frame.dlc = 9U;

    // Act & Assert.
    EXPECT_FALSE(driver::can::isValid(frame));
}
```

Arrange says which starting state the requirement applies in, Act says what is to happen, and **Assert is the requirement**.

Note the values as well: 8 and 9, 0x7FF and 0x800. Not 4 and 0x400. **Bugs live on the boundaries** — a comparison that should be `<=` but was written `<` works for every single value but one.

5.2.5 Reading a failing suite

```
FAILED: EXPECT_EQ(frame.data[0], 0xAAU) at test_frame.cpp:42
```

The procedure, and **without opening your own code in steps 1–3**:

1. Read the **test name**. It says which requirement is broken.
2. Read **Arrange**. It says under which conditions.
3. Read the **Assert line** that failed. It says what was expected.
4. Only then: open your code.

Steps 1–3 take a minute and make step 4 short. Jumping straight to step 4 is the most common reason a simple bug takes an hour.

Note: The supplied test suite is often a **more precise requirements specification** than the prose in the project description. The prose is written by a human and can be ambiguous; the suite compiles and runs. If you are wondering exactly what a method is to return in a corner case: open the test file before you guess.

Summary

- Frame is an **aggregate**: no constructor, no destructor, no methods. That keeps aggregate initialisation readable and makes the class pure data.

- `MaxDataLen` is `static constexpr` — a type, a scope, visible in the debugger, and the same number as the size of the array.
- `isValid()` is a free function, and the rule is expressed **once**. The hardware validates nothing, so the driver has to — and only it.
- You write no tests in the course. You run them, read them and get them to pass.
- Testing decides the course because the hardware neither exists nor is yours until second to last.
- Read a failing test case in the order test name → Arrange → Assert, and open your own code last.
- The test suite is more often a more precise requirements specification than the prose.

Exercises

The solutions to the exercises that do not ask for code are in [appendix A](#). The code is published in the course repository after the lecture.

Exercise 5.1 Predict

REASONING

Answer before you compile, then check.

- What is `sizeof(driver::can::Frame)` on your machine? Why is it not 11?
- Does `driver::can::Frame f{0x123U, 2U, {0xAAU, 0xBBU}}` compile? What does `f.data[7]` become?
- What happens if you add a constructor of your own and then try to compile the line in b)?

Exercise 5.2 constexpr

CODE

Add the following to a test file:

```
static_assert(driver::can::isValid(driver::can::Frame{0x7FFU, 8U, {}}));
static_assert(!driver::can::isValid(driver::can::Frame{0x800U, 0U, {}}));
```

- What does it prove that the lines compile?
- What do they cost at run time?

Exercise 5.3 Read the suite

REASONING

Open the supplied `test_frame.cpp` and answer without running it.

- a) Which boundary values are tested for `id` and for `d1c`? Why those in particular?
- b) Which test case would fail if `isValid()` wrote `<` instead of `<=`? Which would still be green?
- c) Is there any requirement in this chapter that the suite does *not* check?

Exercise 5.4 Break the code

CODE

Change `isValid()` so that the `id` bound becomes `<` instead of `<=`, and run the suite.

- a) Which test case fails, and what does the output say?
- b) Follow the procedure in §5.2. How long did it take to find the error?
- c) Restore the code.

Exercise 5.5 Where the validation sits

REASONING

A group member proposes that `EchoNode` should check `d1c` as well before it calls `send()`, “just to be safe”.

- a) What is the argument for?
- b) What is the argument against?
- c) What happens the day `MaxDataLen` changes?

CHAPTER 6

The interface and the stub

LECTURE 6 · THE CAN DRIVER

The seam that lets the application never need to know where it is.

IN THIS CHAPTER

- `driver::can::Interface`: four methods, and the justification for every `noexcept`, `const` and `[[nodiscard]]`.
- What deliberately is *not* in the interface, and the test that decides it.
- `driver::can::Stub`: a test double in memory.
- Stub, mock, and where the limit of a test double's realism lies.

6.1 The interface

```
namespace driver::can
{
    struct Frame;

    class Interface
    {
    public:
        virtual ~Interface() noexcept = default;

        [[nodiscard]] virtual bool send(const Frame& frame) = 0;
        [[nodiscard]] virtual bool receive(Frame& frame) = 0;
        [[nodiscard]] virtual bool hasError() const noexcept = 0;
        virtual void clearError() noexcept = 0;
    };
} // namespace driver::can
```

Every part of it, one at a time:

A virtual destructor. Objects will be deleted through an `Interface*`. Without `virtual`, only the base class's destructor runs, and a `Spi` holding resources leaks them — undefined behaviour, and one of the few bugs in this design that no test catches. = `default` because there is nothing to do; `noexcept` because destructors must never throw.

= **0 on all four.** The class is purely abstract: it cannot be instantiated and has no implementation to inherit by accident. It is a contract and nothing else.

[[nodiscard]] on three of the four. `send()` returns `false` when the hardware is busy. A caller that ignores that believes the frame went out — the most common driver bug there is, turned into a compiler warning here.

const on hasError(). Asking about an error does not change the driver, and the method can therefore be called on a `const Interface&`.

noexcept on the two error methods. They are called from error-handling paths, and an error-handling path that can itself throw is hard to reason about. They also do nothing that *can* fail.

6.1.1 What is not here

No `init()`. No `setBaudRate()`. No `readRegister()`.

The test is: can both Stub and Spi implement the method meaningfully? `readRegister()` they cannot — the stub has no registers. Such a method would have forced the stub to lie, and forced the application to know that registers exist.

6.2 The stub

Stub is `final`, inherits `Interface`, and simulates CAN in memory:

Method	Contract
The constructor	<code>myLastFrame</code> empty, <code>myInitialized</code> = <code>true</code> , <code>myHasData</code> = <code>false</code> .
<code>send()</code>	<code>false</code> if uninitialised. Otherwise: store the frame, set <code>myHasData</code> , return <code>true</code> .
<code>receive()</code>	<code>false</code> if uninitialised or without data. Otherwise: copy out the frame, clear <code>myHasData</code> , return <code>true</code> .
<code>hasError()</code>	Always <code>false</code> .
<code>clearError()</code>	Does nothing.
<code>inject()</code>	Stores a frame for a later <code>receive()</code> .

The copy and move operations are deleted in the public part. A `Stub` represents a (simulated) hardware resource, and two copies of the same resource are not two resources.

6.2.1 Why inject() is not in the interface

It exists only for the tests' sake: it simulates a frame having arrived on the bus. A `Spi` cannot implement it — it cannot conjure up traffic that does not exist.

The consequence is the pattern that recurs throughout the book:

```

driver::can::Stub stub{};           // concrete type: has inject()
app::EchoNode node{stub, 0x200U};   // takes an Interface&: does not
stub.inject(frame);                 // the test talks to the stub directly
node.run();                         // the component knows nothing about it

```

The test holds the **concrete** type, the component sees only the **interface**.

6.2.2 What the stub does not simulate

No errors. No busy hardware. No queue — a second `send()` overwrites the first. No arbitration, no CRC.

That is deliberate. The stub’s job is to let `EchoNode` run, not to be a CAN simulator.

Note: Knowing where the limit of a test double’s realism lies is a skill in itself. **A stub that grows until it is a second implementation of the hardware is a stub that needs testing itself.** Anybody who wants to test how the driver behaves when the hardware is busy needs a different double, one seam further down — and that arrives in chapter 11.

6.2.3 Stub, not mock

A **stub** answers calls with pre-programmed answers; the test checks what the calling code did with the answers. A **mock** also checks that it was called correctly — the right method, the right arguments, the right number of times.

Stub is a stub. `ScriptedTransport` in chapter 11 is closer to a mock.

6.2.4 The seam has already paid off

The argument for an interface is usually hypothetical. Here it is not: an earlier version of the design reached the registers through **memory-mapped I/O** at `0xFF200000`, which assumes an address bus out to the FPGA fabric that the microcontroller does not have.

Everything above `Interface` survived the change to SPI **unchanged**. The old variant had `Fpga` and volatile pointers where we have `Spi` and `ByteTransport`. Neither of them is wrong; they are two implementations under the same seam.

Summary

- The interface is purely abstract and has four methods. The destructor must be `virtual`, or an implementation deleted through a base class pointer leaks.
- `[[nodiscard]]` on `send()` turns the most common driver bug — an ignored return value — into a compiler warning.
- The test for whether a method belongs in the interface: **can both `Stub` and `Spi` implement it meaningfully?**

- `inject()` therefore lives in `Stub`, not in `Interface`. The test holds the concrete type; the component sees only the interface.
- The stub deliberately simulates neither errors, nor busy hardware, nor a queue. A stub that grows into a second hardware implementation needs testing itself.
- The seam has already paid for itself: the change from memory-mapped I/O to SPI left everything above `Interface` unchanged.

Exercises

The solutions to the exercises that do not ask for code are in [appendix A](#). The code is published in the course repository after the lecture.

Exercise 6.1 Predict the compiler

REASONING

Answer first, compile afterwards.

- You remove `virtual` from the destructor, create a `Stub` with `new` and delete it through an `Interface*`. Does it compile? What goes wrong?
- You remove `[[nodiscard]]` from `send()` and write `driver.send(frame);`. What does the compiler say, with and without the attribute?
- You misspell a method but write `override`. And if you leave out `override`?

Question c) is the reason `override` is mandatory in the project.

Exercise 6.2 One method too many

REASONING

Somebody proposes adding to the interface:

```
[[nodiscard]] virtual std::uint32_t readStatusRegister() const noexcept = 0;
```

"It is handy when debugging."

- What would `Stub` return from it?
- What happens to `EchoNode` if somebody starts using it?
- What happens the day the register map changes?
- Where does the method belong instead?

Exercise 6.3 The polymorphism

CODE

In the supplied `test_stub.cpp` there is a helper function taking a `driver::can::Interface&` instead of a `Stub&`.

- a) Does it contain the word `Stub` anywhere?
- b) Why is it written against the interface?
- c) In chapter 11 the same function is called with a `Spi`. What in your code has to be right for that to work?

Exercise 6.4 The limits of the stub

REASONING

- a) List four things the stub does not simulate.
- b) Pick one of them and describe how you could add it.
- c) What would it cost? When does a test double become so complicated that it needs testing itself?

Component testing through the stub

LECTURE 7 · THE CAN DRIVER

Several parts together, still without hardware.

IN THIS CHAPTER

- The difference from a unit test, with a bug only the component test can see.
- `app::EchoNode`, and dependency injection.
- The layer boundary expressed in a single line of code.
- What the component test still does not prove.

7.1 The bug a unit test cannot see

A **unit test** tests one unit in isolation. A **component test** tests several parts working together, still without hardware: here `app::EchoNode` plus `driver::can::Stub`, connected through `driver::can::Interface`.

The difference shows most clearly in the kind of bug they catch. Suppose `run()` is written like this:

```
Frame frame{};
if (!myDriver.receive(frame)) { return; }

Frame response{};
response.id = myResponseId;
response.dlc = myDriver.receive(frame) ? frame.dlc : 0U; // wrong: calls receive-
↳ () again
```

Every unit test is still green. Stub does exactly what it promises, and so does `Frame`. The bug is in the **interaction**: the second call consumes the state the first one left behind.

That is the whole point. Interaction bugs — one call too many, a call in the wrong order, an ignored return value — are the most common kind of fault in driver code, and they live between the units, not in them.

7.2 app::EchoNode

An application component that does one thing: reads a frame and echoes it back with a different ID.

```
namespace driver::can { class Interface; }

namespace app
{
class EchoNode
{
public:
    explicit EchoNode(driver::can::Interface& driver, std::uint16_t responseId) ↪
        ↪ noexcept;

    void run() noexcept;
    [[nodiscard]] std::uint32_t echoCount() const noexcept;

private:
    driver::can::Interface& myDriver; /**< Driver used for all traffic. Not owned.↪
        ↪ */
    std::uint16_t myResponseId;
    std::uint32_t myEchoCount;
};
} // namespace app
```

7.2.1 The layer boundary, in one line

Note what the header does *not* include: `stub.hpp`, `spi.hpp`, `byte_transport.hpp`. Only a forward declaration.

That line is the architecture's boundary expressed in code. A reviewer who sees `#include "driver/can/spi.hpp"` here should stop the PR on the spot. The rule, which holds for the whole project: no file above `driver/can/interface.hpp` may contain the word SPI.

7.2.2 Dependency injection

`EchoNode` **does not create** its driver. It is given one. The alternative is worth looking at to see what it costs:

```

class EchoNode
{
public:
    EchoNode() : myDriver{} {}    // creates its own Spi
private:
    driver::can::Spi myDriver;    // no
};

```

That variant is impossible to test: there is no way to give it a stub, and therefore no way to run it without an FPGA on the desk. It also binds the application to a concrete driver type, which makes the whole Interface pointless.

Why a reference and not a pointer? A reference cannot be null and cannot be swapped after construction. Both are properties we want.

Why not `std::unique_ptr`? Because `EchoNode` does not *own* the driver. The ownership sits with the factory in chapter 12, and the driver’s lifetime is longer than the component’s. The Doxygen comment explicitly says “Not owned”, and that is worth writing out.

7.2.3 Two details in `run()`

The loop runs to `frame.dlc`, not to `MaxDataLen`. Copying eight bytes would have “worked” since the rest are zeroes anyway — but `response.dlc` says how many apply, and code that copies more than it claims to copy stops being right when some other assumption changes. Exactly the same reasoning as the masking in chapter 10.

`myEchoCount` is incremented only when `send()` succeeded. The counter means “frames echoed”, not “attempts”.

7.3 Reading the component test

```

TEST(EchoNode, echoesInjectedFrameWithResponseId)
{
    // Arrange.
    driver::can::Stub stub{};
    app::EchoNode node{stub, 0x200U};

    driver::can::Frame incoming{};
    incoming.id      = 0x123U;
    incoming.dlc     = 2U;
    incoming.data[0] = 0xAAU;
    incoming.data[1] = 0xBBU;
    stub.inject(incoming);
}

```

```
// Act.
node.run();

// Assert.
driver::can::Frame echoed{};
EXPECT_TRUE(stub.receive(echoed));
EXPECT_EQ(echoed.id, 0x200U);
EXPECT_EQ(node.echoCount(), 1U);
}
```

The test has two roles at once: it **feeds in** traffic with `inject()`, and it **inspects** outgoing traffic with `receive()` — both through the same test double.

What the supplied suite covers:

Case	What it proves
No injected frame	<code>echoCount()</code> unchanged.
One injected frame	The right id, dlc and data are echoed; the counter goes up by one.
Two frames, two <code>run()</code> calls	The counter becomes 2.
One frame, <code>run()</code> twice	The counter becomes 1 — not 2. Catches the bug in §7.1.
<code>dlc = 0</code> and <code>dlc = 8</code>	A pair of boundary values.
<code>send()</code> fails	The counter does not go up.

7.3.1 The same pattern, one layer down

This is the only thing you need to take with you from here. In chapter 11 the supplied suite does **exactly the same thing**, one seam further down: `BankTransport` replaces `Stub`, `Spi` is what is being tested, and the test scripts what the hardware answers instead of which frames come in.

That it *is* the same pattern is no coincidence — it is what you get from putting the seams in the right places.

7.3.2 What it does not prove

That `EchoNode` interacts correctly with *some* implementation of `Interface`. Nothing about `Spi`, which does not exist yet, and nothing about whether the register map has been read correctly.

Summary

- A component test catches **interaction bugs** — one call too many, in the wrong order, with an ignored return value — that every unit test is green in spite of.
- `EchoNode` is given its driver through the constructor. If it creates its own it is impossible to test and `Interface` is pointless.

- A **reference**, not a pointer: cannot be null, cannot be swapped. And not a `unique_ptr`: the component does not own the driver.
- The header forward-declares `Interface` and includes no concrete driver. That line *is* the layer boundary.
- The loop runs to `dlc`, not to `MaxDataLen`. The counter counts successful echoes, not attempts.
- The test feeds in traffic and inspects outgoing traffic through the same double. The same pattern returns one seam further down.

Exercises

The solutions to the exercises that do not ask for code are in [appendix A](#). The code is published in the course repository after the lecture.

Exercise 7.1 Write the bug, watch the test catch it

CODE

Temporarily change `run()` so that it calls `receive()` twice.

- Which test cases fail, and which are still green?
- Run only the unit tests for `Stub` and `Frame`. Does any of them fail?
- Write down in two sentences what the exercise showed.

Exercise 7.2 The layer boundary

REASONING

Does the line break the architecture's layer boundary? Justify in one sentence.

- `#include "driver/can/interface.hpp" in echo_node.hpp.`
- `#include "driver/can/stub.hpp" in test_echo_node.cpp.`
- `#include "driver/can/stub.hpp" in echo_node.cpp.`
- `if (myDriver.hasError()) { myDriver.clearError(); } in run().`
- `static_cast<driver::can::Spi&>(myDriver).readReg(0U); in run().`

Line d) breaks no boundary — but is it a good idea? Who ought to handle errors?

Exercise 7.3 Ownership

REASONING

- What happens if you swap `driver::can::Interface& myDriver` for `driver::can::Interface myDriver`? What does the compiler say, and why?
- What happens if you create an `EchoNode` with a `Stub` that goes out of scope before `run()` is called? Does it compile?
- Why is the driver a reference and not a `std::unique_ptr`?

Exercise 7.4 Read the suite

REASONING

Open the supplied `test_echo_node.cpp`.

- a) Which case would still be green if `run()` forgot to increment the counter?
- b) How does the test get a frame in, and how does it read the echoed one back? Why are those methods not in `Interface`?
- c) Which requirement does the case that calls `run()` twice on one injected frame express?

SPI and the transaction protocol

LECTURE 8 · THE CAN DRIVER

The other half of the contract: how the registers are actually reached.

IN THIS CHAPTER

- SPI on four wires, and why full duplex matters.
- The transaction: one command byte plus four data bytes, always exactly five.
- Latching on a read, commitment on a write, abort on SS.
- What it costs in time, and why the SPI link is the bottleneck.

8.1 SPI on four wires

Wire	Direction	Meaning
SCK	master → slave	Clock. Nothing happens without edges on it.
MOSI	master → slave	The master's data.
MISO	slave → master	The slave's data.
SS	master → slave	Active low. Frames the transaction.

The most important property: **SPI is full duplex**. Every clock pulse simultaneously shifts one bit out on MOSI and one bit in on MISO. There is no way to “just read” — to get a byte in you have to send a byte out.

That is why a read sends four dummy bytes of 0x00, and why the four MISO bytes during a write are meaningless junk.

The parameters: SPI mode 0 (the clock idles low, data is sampled on the rising edge), MSB first, SCK at most 1 MHz.

8.2 The transaction

Every transaction is exactly five bytes: one command byte followed by four data bytes, with SS low the whole way.

Bit:	7	6	5	4	3	2	1	0
	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
	W	0	0	0	register index			
	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+

W = 1 means a write, W = 0 a read. Bits 6–4 are reserved and are to be 0. Bits 3–0 are the register index, 0–12, from the register map in chapter 4.

8.2.1 Writing

The master sends the register value in the four data bytes, **MSB first**. The slave commits the assembled value **only once the fifth byte is complete**.

8.2.2 Reading

At the end of the command byte the slave **latches the register’s current value once**, into its response shifter. The four data bytes then clock that value out.

The latching rule is part of the contract, not an implementation detail. The four bytes always belong to *one* coherent sample of the register, taken in a single instant. Without that rule a STATUS read could have been stitched together from two points in time — the TX-ready bit from before a transmission and the RX-valid bit from after. That would have been a bug appearing about once in a thousand times and never reproducible.

8.2.3 The SS rules

Three rules, and all three have cost somebody an evening:

- **One transaction per low SS period.** The master has to let SS go high between transactions.
- **Back-to-back does not work, and fails silently.** A sixth byte arriving while SS is still low is *discarded* — it is not interpreted as a new command byte. The slave being idle is therefore not enough.
- **An abort has no side effects.** If SS goes high before the fifth byte, the transaction is abandoned entirely: nothing is committed, no trigger fires. That is what makes the slave self-healing after a glitch.

That is why ByteTransport in chapter 9 has `select()` and `deselect()` as methods of their own: the framing is the driver’s decision, not the transport’s.

8.2.4 MISO outside a read

The byte the master clocks out **during the command byte** is meaningless. The slave loads its response shifter at the *end* of the command byte, so what leaves during it is leftovers from last time. **Discard it.**

8.2.5 A complete example

Sending a frame (id 0x123, dlc 2, data AA BB):

```
Write TX_ID      : 81 00 00 01 23
Write TX_DLC     : 82 00 00 00 02
Write TX_DATA_LO : 83 AA BB 00 00
Write TX_DATA_HI : 84 00 00 00 00
Write TX_SEND    : 85 00 00 00 01
Read STATUS     : 00 xx xx xx xx  -> MISO gives 00 00 00 00 during transmission,
                                   then 00 00 00 01 when the port is free again
```

Read off the command byte in every line: 0x81 is the write bit plus index 1, that is, TX_ID. The 0x00 on the last line is a read of index 0.

Note 83 AA BB 00 00: **data byte 0 in the most significant position**. It is the same packing on both sides of the wire.

8.3 What it costs in time

A transaction is 5 bytes = 40 bits. At 1 MHz the clocking takes 40 μ s, plus SS handling. Reckon on **40–50 μ s per transaction**:

Operation	Transactions	Roughly
hasError()	1	45 μ s
receive(), no frame	1	45 μ s
receive(), one frame	6	270 μ s
send()	6	270 μ s

Set that against the CAN side: a maximal CAN frame is around 130 bits, that is, **130 μ s** at 1 Mbit/s.

A send() takes longer than the frame it sends. The SPI link, not the CAN bus, is the bottleneck in this design.

And the polling can miss frames. The controller does not buffer: if two frames arrive close together and your receive() takes 270 μ s, the second has time to overwrite the first. That is the hardware's documented limitation, not your bug — but it is your code that notices it.

Summary

- SPI is **full duplex**: every clock pulse shifts one bit in each direction. A read therefore has to send dummy bytes.

- Every transaction is **exactly five bytes**: a command byte plus four data bytes, MSB first, with SS low the whole way.
- On a read the register value is **latched** once, at the end of the command byte. The four bytes always belong to one coherent sample.
- The byte clocked in during the command byte is junk. Discard it.
- SS has to go high between transactions. Two transactions during one low period fail *silently*.
- A transaction costs 40–50 μs . A `send()` is six of them, that is, longer than the CAN frame it sends.

Exercises

Pen and paper. The solutions are in [appendix A](#).

Exercise 8.1 Decode

DECODING

The following bytes go out on MOSI. For each one: read or write, which register, and which value?

- a) 81 00 00 07 FF
- b) 85 00 00 00 01
- c) 8B 00 00 00 00
- d) C2 00 00 00 03

Two of them are unusual, each in its own way. Which, and why?

Exercise 8.2 Encode

CALCULATION

Write out the complete five-byte transaction for each of the following:

- a) Read RX_DLC.
- b) Write TX_DLC = 4.
- c) Trigger a transmission.
- d) Acknowledge a received frame.
- e) Clear the error register.

Note the difference between d) and e) in particular.

Exercise 8.3 What comes back

REASONING

- a) During a write the master clocks in five bytes on MIS0. How many of them mean anything?
- b) During a read of STATUS, how many of the five mean anything, and which?
- c) A driver assembles the return value from *all five* received bytes instead of the last four. What happens, and what would it look like while debugging?

Exercise 8.4 SS and aborts

REASONING

What happens in each of the cases?

- a) SS goes high after three of five bytes in a write to TX_ID.
- b) The master holds SS low and sends ten bytes in a row: 81 00 00 01 23 82 00 00 00 02.
- c) The master sends the command byte 0x91.

Case b) is the dangerous one. Why, and what would the symptom look like at bring-up?

Exercise 8.5 Time budget

CALCULATION

At 1 MHz SCK:

- a) How long does a `receive()` that finds a frame take?
- b) How long does a maximal CAN frame on the bus take at 1 Mbit/s?
- c) Another node sends frames as fast as it can. What happens to the ones that arrive while your `receive()` is running? Is that your bug?

The transport seam and the register access

LECTURE 9 · THE CAN DRIVER

The protocol becomes code, and the driver becomes runnable on a laptop.

IN THIS CHAPTER

- ByteTransport: why Interface was not enough.
- Register indices, status masks and the command bit as named constants.
- `readReg()` and `writeReg()`, line by line.
- The sign extension bug, which is the most common one in the project.

9.1 Why yet another seam

Interface made Stub and Spi interchangeable, and thereby EchoNode testable. But Spi itself lies *below* that seam. Without something more it can only be run on real hardware.

Work out what that would mean. Spi is finished in chapter 10. The hardware exists in chapter 13. In between you would have an untried class that nobody has run a single time — and the first time it is run, it is run on a board another group is debugging at the same time, over four wires that may or may not be in the right place.

That is not a good way to find a bit-shifting error.

```
namespace driver::can
{
/**
 * @brief Moves single bytes to and from the CAN controller, and frames ↵
 * ↵ transactions.
 *
 *
 *      Knows nothing about registers, command bytes or CAN: it only carries ↵
 * ↵ bytes.
```

```

    */
class ByteTransport
{
public:
    virtual ~ByteTransport() noexcept = default;

    virtual void select() noexcept = 0;    /**< Assert SS; begin a transaction. ↵
    ↵ */
    virtual void deselect() noexcept = 0;  /**< Release SS; end the transaction. ↵
    ↵ */

    [[nodiscard]] virtual std::uint8_t transfer(std::uint8_t byte) noexcept = 0;
};
} // namespace driver::can

```

Three methods. That is the whole layer.

Why `transfer()` returns a byte. SPI is full duplex; a signature like `void transfer(std::uint8_t)` would lie about what the hardware does.

Why `select()/deselect()` are methods of their own. The SS framing is part of the *protocol*, not of how a byte is transferred. The transport should not have to know that a transaction is five bytes long. Keep it dumb, and it does one thing only — and is easy to implement correctly both on the target and in a test.

Why `noexcept` everywhere. The layer runs on a freestanding target without exceptions. Writing that in the signature already now makes it impossible to sneak in something that throws.

9.2 The constants

```

class Spi final : public Interface
{
    // ...
private:
    /** Register indices, as sent in the command byte. */
    static constexpr std::uint8_t RegStatus{0U};
    static constexpr std::uint8_t RegTxId{1U};
    static constexpr std::uint8_t RegTxDlc{2U};
    static constexpr std::uint8_t RegTxDataLo{3U};
    static constexpr std::uint8_t RegTxDataHi{4U};
    static constexpr std::uint8_t RegTxSend{5U};
    static constexpr std::uint8_t RegRxId{6U};
    static constexpr std::uint8_t RegRxDlc{7U};

```

```

static constexpr std::uint8_t RegRxDataLo{8U};
static constexpr std::uint8_t RegRxDataHi{9U};
static constexpr std::uint8_t RegRxAck{10U};
static constexpr std::uint8_t RegErrorFlags{11U};
static constexpr std::uint8_t RegTxAbort{12U};

/** STATUS register bit masks. */
static constexpr std::uint32_t StatusTxReady{1U << 0U};
static constexpr std::uint32_t StatusRxValid{1U << 1U};
static constexpr std::uint32_t StatusError{1U << 2U};

/** Command byte: bit 7 set means write. */
static constexpr std::uint8_t CmdWrite{0x80U};

/** Value written to the write-triggered registers. */
static constexpr std::uint32_t Trigger{0x1U};

ByteTransport& myTransport; /**< Transport used for every register access. Not-
    owned. */
};

```

Indices, not offsets. The offset column of the register map is a legacy of the memory-mapped variant. What goes out on the wire is the index, that is, offset/4. Write the constants as indices directly, and you avoid a division on every call and a class of bugs where somebody divides twice or not at all.

1U << 0U and not 0x01U. The mask is written as a shift because the register map is written in bit positions. StatusRxValid{1U << 1U} can be compared line by line against the table; 0x02U cannot.

9.3 The two helper functions

```

void Spi::writeReg(const std::uint8_t index, const std::uint32_t value) const noexcept
{
    myTransport.select();

    static_cast<void>(myTransport.transfer(static_cast<std::uint8_t>(CmdWrite |
        index)));
    static_cast<void>(myTransport.transfer(static_cast<std::uint8_t>(value >> 24U)));
    static_cast<void>(myTransport.transfer(static_cast<std::uint8_t>(value >> 16U)));
}

```

```

        static_cast<void>(myTransport.transfer(static_cast<std::uint8_t>(value >> 8U)) <-
        );
        static_cast<void>(myTransport.transfer(static_cast<std::uint8_t>(value)));

        myTransport.deselect();
    }

    std::uint32_t Spi::readReg(const std::uint8_t index) const noexcept
    {
        myTransport.select();

        static_cast<void>(myTransport.transfer(index));

        std::uint32_t value{};
        value |= static_cast<std::uint32_t>(myTransport.transfer(0U)) << 24U;
        value |= static_cast<std::uint32_t>(myTransport.transfer(0U)) << 16U;
        value |= static_cast<std::uint32_t>(myTransport.transfer(0U)) << 8U;
        value |= static_cast<std::uint32_t>(myTransport.transfer(0U));

        myTransport.deselect();
        return value;
    }

```

Read `writeReg()` against `81 00 00 01 23` in chapter 8. Four things to note:

Every `static_cast<void>` is deliberate. `transfer()` is `[[nodiscard]]`, and during a write the return value is meaningless. The cast tells both the compiler and the reader that this is known.

In `readReg()` the first byte is discarded. The slave loads its response shifter at the *end* of the command byte. Four `transfer()` calls contribute to the value, not five.

The command byte is just index on a read. Bit 7 low means a read; no `CmdWrite`.

The methods are `const`. They do not change the state of the `Spi` object — the reference itself does not change. That the hardware changes is another matter; `const` in C++ speaks about the object, not about the world. That is why `hasError()` `const` can call `readReg()`.

9.3.1 The sign extension bug

This is the most common bug in the project, and it deserves a heading of its own:

```

value |= myTransport.transfer(0U) << 24U;                                // wrong
value |= static_cast<std::uint32_t>(myTransport.transfer(0U)) << 24U; // right

```

`transfer()` returns a `std::uint8_t`. In the first line it is promoted to `int` — 32 bits *with sign* — and shifted 24 steps. For a value with bit 7 set, that writes into the sign bit. The

result is undefined behaviour, and in practice a negative number that is then converted to `std::uint32_t` with `0xFF` bytes where there should be none.

The symptom: the driver works for small values and breaks as soon as a byte has bit 7 set. So it works for `0x123` and breaks for `0xAABB`.

Rule: Cast **before** the shift, not after. The same rule applies to the packing in chapter 10.

9.3.2 Why the helper functions are private

They are implementation details. Nobody outside the class should be able to poke at the registers, for the same reason that `Interface` has no `readStatusRegister()`: the day somebody does it in the application code, the architecture is broken.

The supplied test suite tests them anyway — through `send()` and `receive()`, and by inspecting which bytes came out. Testing a private method directly is almost always a sign that it ought to be public, or that the test is testing the wrong thing.

9.3.3 What the layer does not do

`readReg()` and `writeReg()` are pure translations of the protocol and contain no logic at all: no validation, no status check, no notion of what a register means.

All the meaning lives in the layer above, and that is chapter 10. The moment `writeReg()` starts containing “if the index is `RegTxSend`, first check that...” two layers have melted together, and both became harder to test.

Summary

- `Interface` makes the application testable; `ByteTransport` makes **the driver** testable. Without the second seam, `Spi` runs for the first time on hardware somebody else is debugging at the same time.
- The transport is deliberately dumb: `select()`, `deselect()`, `transfer()`. It does not know that a transaction is five bytes long — the framing belongs to the protocol.
- The register constants are **indices**, not offsets, and the masks are written as shifts so that they can be compared line by line against the register map.
- `readReg()` discards the byte clocked in during the command byte. Four out of five contribute.
- **Cast before the shift.** Otherwise a `std::uint8_t` is promoted to `int`, and a byte with bit 7 set shifted 24 steps writes into the sign bit.
- The helper functions are private and contain no logic. All the meaning lives in the layer above.

Exercises

The solutions to the exercises that do not ask for code are in [appendix A](#). The code is published in the course repository after the lecture.

Exercise 9.1 Predict

CALCULATION

Answer before you compile.

- What is the command byte for a write to `RegTxDataHi`? For a read of the same?
- Which sequence of five bytes does `writeReg(RegTxDataLo, 0xAABB0000U)` produce?
- What does `readReg()` return if the transport answers FF 12 34 56 78 to the five `transfer()` calls?
- What does it return if you accidentally count all five received bytes?

Exercise 9.2 The sign extension

CODE

Write `readReg()` **wrongly** once, with the shift before the cast. Let the transport answer so that the first data byte is `0x80`.

- What does value become, and why?
- For which values does the incorrect code work anyway?
- What would the symptom look like if the bug were still there at bring-up? Which id values would have worked?

Exercise 9.3 A transport to debug with

CODE

Write a `PrintTransport` that implements `ByteTransport` and prints every call:

```
select
-> 81  <- 00
-> 00  <- 00
-> 00  <- 00
-> 01  <- 00
-> 23  <- 00
deselect
```

Compare the output line by line against the example in chapter 8. It is not a test double but a debugging tool, and it is worth keeping for the whole project.

Exercise 9.4 The layer boundary

REASONING

True or false, with a justification:

- a) `byte_transport.hpp` may include `frame.hpp`.
- b) `ByteTransport` ought to have a method `writeRegister(index, value)`.
- c) `ByteTransport` ought to do the whole five-byte transaction in one call.
- d) `readReg()` ought to be public so that the tests can call it directly.

Statement c) sounds the most reasonable and is wrong anyway. Why?

CHAPTER 10

The driver

LECTURE 10 · THE CAN DRIVER

The four methods, and the limitations they must not hide.

IN THIS CHAPTER

- `send()`: validation, status check, five writes, one trigger — in that order.
- Packing and unpacking across register pairs.
- `receive()`, and why it masks against dlc even though the hardware already zeroes.
- `clearError()`, which writes zero and not one.

10.1 `send()`

```
bool Spi::send(const Frame& frame)
{
    if (!isValid(frame))                { return false; }
    if ((readReg(RegStatus) & StatusTxReady) == 0U) { return false; }

    writeReg(RegTxId,      frame.id);
    writeReg(RegTxDlc,     frame.dlc);
    writeReg(RegTxDataLo,  packDataLo(frame));
    writeReg(RegTxDataHi,  packDataHi(frame));
    writeReg(RegTxSend,    Trigger);

    return true;
}
```

10.1.1 The order is not arbitrary

`TX_SEND` **must** be written last. That write does not mean “store a value” but “start transmitting now”, and the controller then reads `TX_ID`, `TX_DLC` and the two data registers. Write the

trigger first and you send the previous frame’s contents — which at bring-up looks like a node running exactly one frame behind.

The status check has to come **before** the writes. If you were to write `TX_ID` while a transmission is in progress, you would overwrite data the controller is in the middle of using.

10.1.2 Two falses, two meanings

The interface has only one `bool` to express both with:

Cause	What the caller ought to do
Invalid frame	Fix the frame. Waiting never helps.
Hardware busy	Wait and try again.

That is a deliberate simplification, but one worth being aware of. Two things follow:

The order matters. The validation first, so that an invalid frame is rejected *without* a single transaction going out on the bus. The supplied test suite checks exactly that.

It becomes a problem exactly one seam further up. This loop is an infinite loop for a frame with `dlc = 9`:

```
while (!driver.send(frame)) // hangs forever if the frame is invalid
{
}
```

The right handling is to validate before entering the loop, or to count attempts and give up.

10.2 The packing

Data byte 0 sits in the **most significant** bits, and `TX_DATA_L0` carries the **first** four bytes.

```
/**
 * @brief Pack four payload bytes into one register word, byte 0 in the most
 *        significant bits.
 */
[[nodiscard]] std::uint32_t pack(const std::uint8_t* const data,
                                const std::uint8_t offset) noexcept
{
    return (static_cast<std::uint32_t>(data[offset + 0U]) << 24U)
        | (static_cast<std::uint32_t>(data[offset + 1U]) << 16U)
        | (static_cast<std::uint32_t>(data[offset + 2U]) << 8U)
        | (static_cast<std::uint32_t>(data[offset + 3U]));
}
```

The same cast rule as in chapter 9: **cast before the shift**.

The two most common packing errors:

Error	Symptom
TX_DATA_LO and TX_DATA_HI swapped	The frame arrives with bytes 4–7 first. Shows immediately in a bus analyser, and not at all in simulation if the test is wrong too.
The shifts in reverse order	AA BB CC DD arrives as DD CC BB AA.

Both give a frame that *does arrive* — just with the wrong content. That is the most unpleasant kind of error, because everything *looks* like it works.

10.3 receive()

```
bool Spi::receive(Frame& frame)
{
    if ((readReg(RegStatus) & StatusRxValid) == 0U) { return false; }

    frame.id = static_cast<std::uint16_t>(readReg(RegRxId) & 0x7FFU);
    frame.dlc = static_cast<std::uint8_t>(readReg(RegRxDlc) & 0xFU);

    unpack(readReg(RegRxDataLo), frame.data, 0U);
    unpack(readReg(RegRxDataHi), frame.data, 4U);

    for (std::uint8_t i{frame.dlc}; i < Frame::MaxDataLen; ++i)
    {
        frame.data[i] = 0U;
    }

    writeReg(RegRxAck, Trigger);
    return true;
}
```

10.3.1 The masking against dlc

The loop looks redundant. The register map does say that the controller clears its data accumulator at the start of every frame, so those bytes *are* already zero.

Do it anyway, for three reasons:

1. **The zeroes are the controller's guarantee, not the frame's content.** A driver that copies eight bytes straight off reports up to six bytes that were never sent. That they happen to be zero does not make them data.

2. **The guarantee can disappear.** It comes from an implementation detail in the hardware's VHDL. If that changes, or if you borrow a board from another group, your driver is still right.
3. **d1c is the contract.** `frame.d1c = 2` means "two bytes apply". Code that hands out an object whose fields contradict each other leaves the next reader unsure.

10.3.2 RX_ACK last

The acknowledgement releases the `RX_*` registers and clears STATUS bit 1. Write it **after** you have read everything — otherwise the next frame can overwrite the registers in the middle of your reading.

Forget it entirely, and bit 1 stays set forever: every `receive()` returns `true` with the same frame, in an endless loop. That is a symptom worth recognising.

10.4 `hasError()` and `clearError()`

```
bool Spi::hasError() const noexcept
{
    return (readReg(RegStatus) & StatusError) != 0U;
}

void Spi::clearError() noexcept
{
    writeReg(RegErrorFlags, 0U);
}
```

Two lines, and one of them is a trap.

`clearError()` writes `0x0`, not `0x1`. `ERROR_FLAGS` is a latch-and-clear-on-zero construction: only a write where the whole word is zero clears the latch. `0x1` does nothing, and `0x000000FF` does nothing even though bit 0 is set.

That is the opposite rule to `TX_SEND` and `RX_ACK`, which require bit 0 to *be* set. That the three registers have different rules is not elegant, but it is what the map says, and it is the map that counts.

The symptom if you write `0x1`: the error bit never goes low, `hasError()` returns `true` forever after the first error, and all error handling above stops working.

10.5 Waiting for a transmission

`send()` **does not wait**. It returns as soon as the trigger is written, and that is right: a driver method that blocks for up to 130 μ s on a 4 MHz MCU eats the whole system's time budget.

So how does the caller wait? The interface has neither `txReady()` nor `waitForSend()` — deliberately, since `Stub` could not have implemented them meaningfully. What the caller has is that `send()` returns `false` while the transmitter is busy. **Waiting is therefore another attempt:**

```
driver.clearError();

if (!isValid(frame)) { return false; }    // never going to be accepted, however ↪
    ↪ long we wait

std::uint16_t attempts{};
while (!driver.send(frame))
{
    if (++attempts >= MaxAttempts) { return false; }
}

// The port was free when the frame was accepted, so the previous transmission had↪
    ↪ finished.
if (driver.hasError())
{
    driver.clearError();
    // Lost arbitration, stuff error, bad CRC or a remote frame - the latch does ↪
    ↪ not say which.
}
```

A lost arbitration looks like a successful transmission. STATUS bit 0 comes back in both cases, and means *the port is free*, not *the frame arrived*. If you want to know the outcome you have to read `ERROR_FLAGS` *after* the bit has come back — and even then you learn *that* something went wrong, never *what*.

10.6 Checklist

- ☐ `send()` validates **first**, and sends no transaction for an invalid frame.
- ☐ `send()` checks `StatusTxReady` before it writes anything.
- ☐ `TX_SEND` is written **last**.
- ☐ `TX_DATA_L0` carries data bytes 0–3, with byte 0 in bits 31–24.
- ☐ Every `static_cast<std::uint32_t>` stands **before** the shift.
- ☐ `receive()` checks `StatusRxValid` first.
- ☐ `receive()` writes `RX_ACK` **after** all the reads.
- ☐ `receive()` zeroes `frame.data` above `frame.dlc`.
- ☐ `clearError()` writes `0x0`.

□ Not a single magic number in the whole file.

Summary

- `send()`: validate, check the status, write four data registers, trigger **last**. The trigger means “start transmitting now”, and the controller then reads the other registers.
- `send()` returns `false` for two reasons, and only one of them goes away by itself. `while (!send(frame)) {}` hangs forever on an invalid frame.
- `TX_DATA_LO` carries data bytes 0–3. Swapping them gives a frame that arrives back to front — everything *looks* like it works.
- `receive()` masks against dlc even though the hardware already zeroes: the zeroes are the controller’s guarantee, not the frame’s content.
- `RX_ACK` is written last. Forget it and you read the same frame in an endless loop.
- `clearError()` writes `0x0`. Write `0x1` and the error bit never goes low.
- `send()` does not wait. Waiting is another attempt, and happens at the caller.

Exercises

The solutions to the exercises that do not ask for code are in [appendix A](#). The code is published in the course repository after the lecture.

Exercise 10.1 Break the code on purpose

CODE

Make one change at a time, run against your `PrintTransport`, and note what the output shows and how the error would manifest on a real bus. Restore after each one.

- `TX_SEND` is written first instead of last.
- `TX_DATA_LO` and `TX_DATA_HI` swap places.
- The shifts inside `pack()` in reverse order.
- `clearError()` writes `0x1`.
- `receive()` forgets `RX_ACK`.
- `receive()` does not mask against dlc.
- `send()` skips the status check.

Line f) is the hardest: the output *looks right*. Why, and which test case in chapter 11 catches it anyway?

Exercise 10.2 Count transactions

CALCULATION

Write it down without running it, then check.

- a) A `send()` that succeeds?
- b) A `send()` that returns `false` because the hardware is busy?
- c) A `send()` that returns `false` because `dLC = 9`?
- d) A `receive()` that finds no frame?
- e) A `receive()` that finds one?

Question c) is the point: the validation comes first *precisely so that* an invalid frame costs not a single transaction.

Exercise 10.3 The infinite loop

CODE

- a) Run `while (!driver.send(frame)) {}` with a valid frame against a transport that says the hardware is busy. What happens?
- b) Run the same loop with `frame.dLC = 9`. What happens?
- c) Rewrite the loop so that both cases are handled.

Exercise 10.4 Design questions

REASONING

Short answers; they belong in the project report.

- a) `send()` returns `false` for two completely different reasons. What would you have changed in `Interface` to tell them apart, and what would it have cost `Stub`?
- b) Give a scenario where the masking against `dLC` actually makes a difference.
- c) Your node transmits a frame and loses the arbitration. Describe step by step what the driver observes — and what it *cannot* observe.
- d) The controller does not buffer received frames. Which change in *your* code reduces the risk of losing a frame the most, and which limit will you still not get past?

Testing without hardware

LECTURE 11 · THE CAN DRIVER

Two test doubles, two questions, and the whole stack on a laptop.

IN THIS CHAPTER

- ScriptedTransport: did the driver send the right bytes?
- BankTransport: does the driver behave correctly?
- Which bug one of them catches and the other misses.
- What a green test suite does not prove.

11.1 Why two

ByteTransport is a small interface — three methods — and precisely for that reason it can be implemented in two completely different ways, answering two different questions.

Double	The question	What it is
ScriptedTransport	<i>Did the driver send the right bytes?</i>	A tape recorder: plays back pre-programmed answers, records everything that was sent.
BankTransport	<i>Does the driver behave correctly?</i>	A simulator: a register bank in memory, with the hardware’s semantics.

Both are supplied. They do not overlap, and both are needed:

Bug	Scripted	Bank
clearError() writes 0x1	Yes, directly	Yes, indirectly
receive() does not mask against dlc	No	Yes
TX_SEND written before the data registers	Yes	No
TX_DATA_LO/HI swapped	Yes	No

Rows two and three are the reason one of them is not enough.

11.2 ScriptedTransport

It does three things: plays back MISO bytes from a script, records every MOSI byte, and counts `select()/deselect()` pairs.

```
TEST(Spi, writeRegSendsCommandByteThenValueMsbFirst)
{
    // Arrange.
    ScriptedTransport transport{};
    driver::can::Spi driver{transport};

    driver::can::Frame frame{};
    frame.id      = 0x123U;
    frame.dlc     = 2U;
    frame.data[0] = 0xAAU;
    frame.data[1] = 0xBBU;

    // Act.
    static_cast<void>(driver.send(frame));

    // Assert: the TX_DATA_LO transaction.
    const auto& sent{transport.sent()};
    EXPECT_EQ(sent[10], 0x83U); // CmdWrite | RegTxDataLo
    EXPECT_EQ(sent[11], 0xAAU);
    EXPECT_EQ(sent[12], 0xBBU);
    EXPECT_EQ(sent[13], 0x00U);
    EXPECT_EQ(sent[14], 0x00U);
}
```

Read that as the protocol, not as code. The five lines in the Assert part are 83 AA BB 00 00 from chapter 8, written as a requirement. That whole test file is in practice the SPI protocol expressed in C++ — and thereby a more precise requirements specification than the prose.

11.3 BankTransport

The more ambitious one. It interprets the command byte, holds thirteen registers in memory, and imitates the hardware's semantics: the masking of TX_ID/TX_DLC, that a write to TX_SEND clears TX ready, that RX_ACK clears RX valid, and that only an all-zero word clears ERROR_FLAGS.

Plus a few methods only the test uses, to simulate what the controller does:

```

void completeTransmission() noexcept;           /**< TX ready goes high again. ↵
↳ */
void deliverFrame(const driver::can::Frame& frame) noexcept; /**< RX valid goes ↵
↳ high. */
void raiseError() noexcept;                     /**< The controller latches an ↵
↳ error. */

```

With those a test case can script a whole sequence of events — for example a frame with `dlc = 2` following one with `dlc = 8`, which is exactly the case the masking loop in chapter 10 exists for.

11.4 The whole stack on a laptop

```

TEST(EchoNode, echoesThroughTheRealDriver)
{
    // Arrange.
    BankTransport bank{};
    driver::can::Spi driver{bank};
    app::EchoNode node{driver, 0x200U};

    driver::can::Frame incoming{};
    incoming.id      = 0x123U;
    incoming.dlc     = 2U;
    incoming.data[0] = 0xAAU;
    incoming.data[1] = 0xBBU;
    bank.deliverFrame(incoming);

    // Act.
    node.run();

    // Assert.
    EXPECT_EQ(node.echoCount(), 1U);
    EXPECT_EQ(bank.readRegister(1U), 0x200U); // TX_ID
    EXPECT_EQ(bank.readRegister(3U), 0xAABB0000U); // TX_DATA_L0
}

```

Compare with the test case in chapter 7. It is **the same test case**, with `Stub` swapped for `Spi + BankTransport`. `EchoNode` is unchanged.

That is the architecture paying you back: the seam in `Interface` made the swap possible, and the seam in `ByteTransport` made it possible *without hardware*.

11.5 From a failing test case to the bug

```
FAILED: EXPECT_EQ(sent[11], 0xAAU) at test_spi.cpp:87
```

The procedure, and **without opening your own code in steps 1–3**:

1. **The test name.** `writeRegSendsCommandByteThenValueMsbFirst` — the requirement is about how a value is written out, MSB first.
2. **The index.** `sent[11]` is the second byte of the third transaction. Count: 0–4 is the STATUS read, 5–9 is TX_ID, 10–14 is TX_DLC... or TX_DATA_L0? Miscounting here is common; print the whole recording and look.
3. **The expected value.** `0xAA` is `frame.data[0]`. The requirement is therefore that data byte 0 ends up first.
4. **Now**, and only now: your `pack()`. Is `data[0]` in bits 31–24?

11.6 What a green suite does not prove

This is more important than it sounds.

A green suite proves that your code agrees with **the test author’s interpretation of the contract**. It does not prove:

- That the **interpretation** is right. The test suite is written by a human who read the same document as you.
- That the **hardware group** implemented the same interpretation. Their testbenches are written against the same document, by a different human.
- That the **wiring** is right. A swapped MOSI/MISO is invisible to every test on the laptop.
- That the **time budget** holds. `BankTransport` answers immediately; a real SPI link does not.
- That you **poll often enough** not to lose frames.

All of that is caught only by chapter 13 and chapter 14. But everything *below* those points is to be green before you connect anything: **a green suite does not make bring-up easy; it makes it possible**, by removing a couple of hundred sources of error before you stand at the bench.

Summary

- Two test doubles, two questions. `ScriptedTransport` checks *which bytes* were sent; `BankTransport` checks *how the driver behaves*.
- They do not overlap. A missing mask against dlc is caught only by the one, a misordered trigger only by the other.

- The ScriptedTransport file is the SPI protocol written as code — and thereby a more precise requirements specification than the prose.
- With BankTransport the **whole stack** runs on the laptop. The test case is the same as in chapter 7, with Stub swapped out. EchoNode is unchanged.
- Read a failing test case in the order test name → index → expected value → your code.
- A green suite proves that the code agrees with the test author's *interpretation*. The wiring, the time budget and the other group's interpretation are outside it.

Exercises

The solutions to the exercises that do not ask for code are in [appendix A](#). The code is published in the course repository after the lecture.

Exercise 11.1 Get the suite green

CODE

Add the supplied files unchanged and run them. Note how many test cases fail the first time.

Work through them one at a time, and keep a log:

Test case	What the name said	Where the bug was	Minutes
-----------	--------------------	-------------------	---------

The “minutes” column tends to fall dramatically as the exercise goes on, and the reason is that you stop guessing.

Exercise 11.2 Which double catches what?

REASONING

For each one: ScriptedTransport, BankTransport, both, or neither? Justify in one sentence, and **then check by introducing the bug**.

- TX_DATA_L0 and TX_DATA_HI swapped.
- clearError() writes 0x1.
- receive() does not mask against d1c.
- receive() forgets RX_ACK.
- readReg() counts the command byte's MIS0 answer.
- send() does not validate d1c.
- MOSI and MIS0 swapped in the wiring.

Row g) is there for a reason. Which?

Exercise 11.3 Read the suite as a requirements specification

REASONING

Open the supplied `test_spi.cpp`.

- a) Which test case corresponds to the example 83 AA BB 00 00 in chapter 8?
- b) Which test case checks that an invalid frame costs not a single transaction? How does it check that?
- c) How many test cases use each double? Does the distribution match the reasoning in §11.1?

Exercise 11.4 The whole stack

CODE

Run the test case connecting `EchoNode` → `Spi` → `BankTransport`.

- a) How many lines have changed in `EchoNode` since chapter 7?
- b) Which property of the architecture made the swap free?
- c) What would have had to change if `EchoNode` had taken a `driver::can::Spi&` in its constructor?

Exercise 11.5 What remains

REASONING

Write a list of everything that can still be wrong even though the whole suite is green. Aim for at least five points, of which at least one you found on your own.

The list is your checklist for chapter 13.

The hardware, the transport and the factory

Down to the bottom: real registers, and the glue that ties it all together.

IN THIS CHAPTER

- **volatile**: what it is for, and where it may appear.
- The microcontroller's SPI peripheral, and the three settings that are traps.
- MVIO: how the core runs at 5 V while the SPI wires run at 3.3 V.
- The factory, and the test for whether the architecture holds.

12.1 volatile

So far the code has only touched ordinary memory. This file touches **hardware**, and at that point one of the compiler's fundamental assumptions stops holding.

The compiler assumes that nobody but the program changes a variable. On that assumption the following loop is infinite, and the optimiser is allowed to remove it:

```
while ((SPI0.INTFLAGS & SPI_IF_bm) == 0U)
{
}
```

INTFLAGS is read once, the result is put in a register, and since nothing in the loop changes it the condition is constant. With `-O2` the result is either a loop that never ends or a loop that is optimised away entirely.

But INTFLAGS *does change* — by the peripheral, without the program doing anything. **volatile** is how you say that: *re-read this variable every time; do not merge reads, do not move them, do not remove them.*

In `<avr/io.h>` the peripheral registers are already declared `volatile`. You do not have to write the qualifier — but you do have to know that it is there, and why.

12.1.1 What `volatile` is not

It is **not** a synchronisation mechanism. It guarantees neither atomicity nor any ordering against anything but other `volatile` accesses. In threaded code on a host computer `volatile` is almost always the wrong tool; `std::atomic` is the right one.

Here it is about something else: a register that a peripheral changes under the program's nose. That is precisely the case `volatile` exists for.

12.1.2 Where it may appear

Only in the transport's implementation file.

That is the whole point of the seam in chapter 9. Everything above it — `Spi`, `Stub`, `Interface`, `EchoNode` — is ordinary, portable C++ that compiles and runs on your laptop. A reviewer who sees `volatile` in any other file should ask why.

12.2 The SPI peripheral

Four registers are enough for a polled master: one for enable and clock settings, one for the mode, one for status flags, and one for data.

```
void AvrSpiTransport::init() noexcept
{
    // Route SPI0 to PORTC (ALT1): PC0 = MOSI, PC1 = MISO, PC2 = SCK, PC3 = SS.
    PORTMUX.SPIROUTEA = PORTMUX_SPI0_ALT1_gc;

    // MOSI, SCK and SS are outputs; MISO is an input. SS idles high.
    PORTC.DIRSET = PIN0_bm | PIN2_bm | PIN3_bm;
    PORTC.DIRCLR = PIN1_bm;
    PORTC.OUTSET = PIN3_bm;

    // Master, MSB first, 4 MHz / 4 = 1 MHz, enabled.
    SPI0.CTRLA = SPI_MASTER_bm | SPI_PRESC_DIV4_gc | SPI_ENABLE_bm;

    // SPI mode 0, and software-controlled slave select.
    SPI0.CTRLB = SPI_MODE_0_gc | SPI_SS0_bm;
}
```

Five lines, and three of them are traps:

The pin mux. Without that line the peripheral sits on its default mux, and the four wires you connected are completely silent. The symptom is the most confusing there is: the program runs, no errors are reported, and the logic analyser shows nothing at all. **Set it first.**

Slave select disable. Without it the peripheral monitors the SS pin, and a low level there makes the master believe that another master has taken over the bus: it leaves master mode. But here it is *your own code* that pulls SS low on every `select()`. Without the bit, the driver shoots itself in the foot on the first transaction.

The prescaler. The protocol guarantees nothing above 1 MHz. Do not run faster just because you can.

12.2.1 The transport

```
void AvrSpiTransport::select() noexcept { PORTC.OUTCLR = PIN3_bm; }
void AvrSpiTransport::deselect() noexcept { PORTC.OUTSET = PIN3_bm; }

std::uint8_t AvrSpiTransport::transfer(const std::uint8_t byte) noexcept
{
    SPI0.DATA = byte;

    while ((SPI0.INTFLAGS & SPI_IF_bm) == 0U)
    {
    }

    return SPI0.DATA;
}
```

Reading SPI0.DATA clears the flag. Skip it and the next `transfer()` sees a flag that is still set, returns immediately, and reads a byte that has not had time to arrive. The symptom is data lagging **one byte behind** through the whole transaction, which is hard to see and easy to mistake for a bit-order error.

No delay is needed around SS. The protocol requires 60 ns of setup and hold; a single port write on a 4 MHz MCU is already several times slower than that.

12.3 MVIO: two voltages in the same chip

This is the most elegant detail in the whole design.

The problem: the MCU board runs at 5 V. The FPGA board's GPIO is 3.3 V and does not tolerate 5 V. The usual solution is a level shifter on every wire — one more component, four more connections, and four more things that can be in the wrong place.

The solution: the AVR-DB family has **MVIO** (*Multi-Voltage I/O*). PORTC is not fed from VDD like the rest of the chip, but from a pin of its own: VDDI02.

Connect VDDI02 to the FPGA board's 3.3 V rail, and the whole of PORTC becomes a 3.3-volt domain — while the core and the other ports run unchanged at 5 V. **No level shifter.**

That is why the pin configuration is part of the contract and not each group’s own choice: switch to the default mux and the wires end up in the VDD domain, and then the level shifter is needed again — or the FPGA input breaks.

Note: VDDIO2 has to be powered **before** PORTC does anything at all. If it is not, the port’s behaviour is undefined. The MVIO status register says whether it is, and checking it at start-up is the single most valuable line of code ahead of bring-up: a forgotten VDDIO2 connection looks exactly like a broken SPI implementation, and that check tells the two cases apart in a second.

12.4 Freestanding C++

The firmware is built for a **freestanding** target. Part of C++ does not come along:

Not available	What you do instead
The heap (new, std::vector)	Statically allocated objects; everything has a known size.
Exceptions	Return codes — which the whole interface is already built on.
RTTI (dynamic_cast)	Virtual functions are enough; you never cast downwards.
<iostream>	UART output, or an LED.

Note that none of that affects the design. The interface returns bool instead of throwing. Frame is an aggregate with a static size. No class uses the heap. That was not luck — the project’s coding conventions are written for a freestanding target from the start, and this is the chapter where that pays off.

12.5 The factory

Every part now exists. What is missing is the glue:

EchoNode	needs an	Interface&
Spi	IS an	Interface, and needs a ByteTransport&
AvrSpiTransport	IS a	ByteTransport

Somebody has to create the objects, in the right order, and keep them alive as long as the program runs. That somebody should not be main, and definitely not EchoNode.

```
class Factory
{
public:
    Factory() noexcept;

    Factory(const Factory&)           = delete;
    Factory(Factory&&)                = delete;
```

```

Factory& operator=(const Factory&) = delete;
Factory& operator=(Factory&&)      = delete;

/**
 * @brief Get the CAN driver.
 *
 * @return Reference to the driver, valid for the lifetime of the factory.
 */
[[nodiscard]] driver::can::Interface& canDriver() noexcept;

private:
    AvrSpiTransport myTransport; /**< Byte transport. Declared before the driver ↵
    ↵ that uses it. */
    driver::can::Spi myDriver;    /**< The CAN driver, bound to myTransport. */
};

```

The declaration order is a requirement, not style. Members are constructed in the order they are declared, and `myDriver` takes a reference to `myTransport` in its constructor. Swap the two lines and you bind a reference to an object that has not been constructed yet. The compiler says nothing.

The return type is `Interface&`, not `Spi&`. That is the whole point. The caller gets something it can use, not something it can recognise.

No heap. The objects are members, the factory sits on the stack in `main`, and everything has a static size. On a freestanding target there is no `std::unique_ptr` to reach for — nor is one needed. Smart pointers in a factory example solve an ownership problem that does not exist when the number of objects is known at compile time.

12.6 main

```

int main()
{
    Factory factory{};
    app::EchoNode node{factory.canDriver(), ResponseId};

    while (true)
    {
        node.run();
    }
}

```

Five lines. `main` does three things and nothing more: creates the factory, creates the application component with what the factory hands out, and runs.

The words `Spi`, `AvrSpiTransport`, `SPI0` and `P0RTC` do not appear. **If you can read your main without knowing which hardware the program runs on, you have got the architecture right.**

12.6.1 The test of the design

Change one line in the factory:

```
private:
    driver::can::Stub myDriver;    // was: AvrSpiTransport + Spi
```

Compile for the **host computer**. It should work, and the program should run — `EchoNode` echoes frames against an array in memory instead of against a CAN bus.

If it does not, the compiler error points out exactly where some line above the interface (`driver/can/interface.hpp`) has learnt something about the hardware that it should not know.

It is a test you can run in ten seconds, and it says more about the quality of the architecture than any code review.

12.7 An application you can debug with

`EchoNode` is good for proving the architecture and bad for debugging with: it does nothing visible. Ahead of chapter 13 it is worth building one more.

LED	Lights when
Green	A frame has been received and acknowledged.
Yellow	A frame has been sent (<code>send()</code> returned <code>true</code>).
Red	<code>hasError()</code> returned <code>true</code> .

Plus a button that sends a frame with a known ID and a known payload.

That is a quarter of an hour's work and it pays for itself immediately at the bench: a red LED lighting up when you connect the transceiver says something completely different from a node that does nothing at all.

Keep it outside `EchoNode`. The LEDs belong to the application, not to the driver, and the driver is to remain buildable for the host computer.

Summary

- `volatile` tells the compiler not to optimise away reads of something the hardware changes. It is **not** a synchronisation mechanism.

- `volatile` may appear in **one single file**: the transport's implementation. That is what the seam in chapter 9 bought.
- Three settings are traps: the pin mux (without it the wires are silent), slave select disable (without it the master leaves master mode on the first `select()`), and the prescaler.
- `transfer()` has to **read** the data register after every transfer. Otherwise all the data is one byte out.
- `MVIO` lets `PORTC` run at 3.3 V while the core runs at 5 V — no level shifter. `VDDIO2` has to be powered before the port does anything.
- The factory's members are declared in construction order: the transport before the driver that binds a reference to it.
- The test of the architecture: swap `Spi` for `Stub` in the factory and compile for the host computer. If it fails, some line above the interface has learnt too much.

Exercises

The solutions to the exercises that do not ask for code are in [appendix A](#). The code is published in the course repository after the lecture.

Exercise 12.1 Watch the optimiser work

CODE

On the host computer, before you write any hardware code:

```
int flag{};

void wait()
{
    while (flag == 0)
    {
    }
}
```

- Compile with `-O2 -S` and look at the assembly. What became of the loop?
- Add `volatile` to `flag` and do it again. What changed?
- Explain in two sentences why the difference matters for the status flag in `transfer()`.

Exercise 12.2 Predict, change, observe

CODE

Make one change at a time and note the symptom. Restore after each one.

- Remove the pin mux line.
- Remove slave select disable from the mode register.
- Set the prescaler so that the clock becomes twice as fast.
- Remove the read of the data register in `transfer()`.

e) Forget to set the clock pin as an output.

Line d) is the most insidious: it *looks* like it works. What actually goes wrong, and when is it noticed?

Exercise 12.3 The architecture test

CODE

Swap the transport and the driver for a Stub in the factory and compile for the host computer.

- a) Did it work?
- b) If not: which file did the compiler complain about, and which layer boundary had been broken?
- c) Run the program. What does EchoNode do against a Stub?

Exercise 12.4 Freestanding

CODE

Build the firmware with `-fno-exceptions -fno-rtti`.

- a) Does everything compile?
- b) Search the whole project for `new`, `std::vector`, `std::string` and `dynamic_cast`. Do you find anything?
- c) Why did the design not have to change for the freestanding target?

Exercise 12.5 The construction order

REASONING

Swap the factory's two member declarations.

- a) Does it compile? Does the compiler warn?
- b) What happens at run time, and why?
- c) Which general rule about member order follows from that?

Bring-up

LECTURE 13 · THE HARDWARE

Two halves, four wires, and a ladder with six rungs.

IN THIS CHAPTER

- The wiring: SPI between the boards, and the CAN side out onto a real bus.
- The bring-up ladder, each rung with a receipt of its own.
- Troubleshooting: from symptom to hypothesis to measurement.
- Debugging something two groups wrote one half each of.

13.1 The wiring

13.1.1 SPI between the boards

Four wires plus power and ground, according to the contract's pin configuration. Two things are worth spelling out:

VDDIO2 first. P0RTC is the MVIO domain and is fed from there. Without that connection the port does nothing at all, and the symptom looks exactly like a broken SPI implementation.

A common ground is not optional. Two boards without a common ground do not communicate, and the symptom is incomprehensible.

13.1.2 The CAN side

The FPGA node's CAN side is a **single-wire open-drain bus** on three GPIO pins: `tx_bus` (what the node drives), `bus_en` (whether it drives at all) and `rx_bus` (the bus level coming in).

For a bus analyser to see the traffic, that single wire has to become a real **differential CAN** bus. That is the transceiver's job, and its transmit input is to be **low for a dominant bit**:

bus_en	tx_bus	The node	Transceiver input
0	—	does not drive	1 (recessive)
1	0	dominant	0
1	1	recessive	1

How the two signals are combined onto the single wire is the hardware side's part of the wiring; check it before you connect anything. A node whose `bus_en` has been ignored drives the bus all the time and blocks everybody else.

Termination: 120 Ω at each end, not in the middle and no more than two. A badly terminated bus often works anyway on a short cable, and stops working precisely when you have started to trust it.

13.2 The ladder

Climb it in order. Every rung has a receipt of its own, and the point is to **never have more than one unknown at a time**. Skip a rung and you are debugging four layers at once.

Rung	Do	Receipt
0	Power and ground	The MVIO status reports that VDDI02 is within range.
1	MOSI to MISO, without the FPGA	You get back exactly the byte you sent. Try 0x00, 0xFF, 0xAA, 0x55.
2	Connect the FPGA; write and read TX_ID	0xFFFFFFFF reads back as 0x000007FF.
3	Read STATUS repeatedly	0x00000001 — TX ready set, the others low.
4	Connect the transceiver; run a <code>send()</code>	Activity on the bus; STATUS bit 0 goes low and comes back.
5	Let a second node transmit	<code>receive()</code> returns true with the right content.

Two of the rungs deserve a comment.

Rung 1 is `AvrSpiTransport`'s only test. It is the only class in the project that no automated test covers, since it talks to a physical register. If the loopback does not work, the fault is in your code or in the configuration — the FPGA is not connected and cannot be the cause.

Rung 2 is the decisive one. The masking to eleven bits proves that you are talking to the *register bank* and not to an echo of your own wire. A loopback gives back what you sent; a register bank gives back what it stored.

13.3 Troubleshooting

Symptom	Likely cause	Check
Nothing at all, no clock pulses	The pin mux is not set	Rung 1
The port does nothing	VDDIO2 not powered	The MVIO status
Every read gives 0x00000000	MISO not connected, or the wrong pin	Swap MOSI/MISO and see whether it changes
Every read gives 0xFFFFFFFF	MISO floating	The wiring
The reads are one byte out	The data register is not read in <code>transfer()</code>	chapter 12
Right for small numbers, not large	Sign extension in <code>readReg()</code>	chapter 9
The second transaction is ignored	Two transactions during one low SS	chapter 8
The frame arrives back to front	TX_DATA_L0/HI swapped	chapter 10
The same frame over and over	RX_ACK is not written	chapter 10
<code>hasError()</code> always true	<code>clearError()</code> writes 0x1	chapter 10
The bus constantly dominant	A node's <code>bus_en</code> ignored	The wiring table
Works on a short cable, not a long	Termination	120 Ω at each end

13.3.1 The procedure

1. Which rung was the last one to give the right receipt?
2. What is the **one** thing that has been added since then?
3. Formulate a hypothesis about what is wrong.
4. Decide which measurement would settle the matter.
5. Make the measurement.

Note: Swapping wires until it works is not troubleshooting, and it takes longer. The difference between the two ways of working is about an afternoon per fault, and it is the most transferable skill in the whole course.

13.3.2 Debugging somebody else's half

A bug during bring-up can be in your code, in the other group's VHDL, or in a jumper wire. Three rules make it manageable:

- **Suspect your own half first.** Not out of humility, but because you can change it and measure on it.
- **Prove it before you report it.** "It does not work" is not a fault report. "Rung 2 gives 0x000007FF for TX_ID but 0x00000000 for TX_DLC" is.
- **Change one thing at a time, and write down what.** Two people debugging the same wiring at the same time make each other's measurements meaningless.

Summary

- VDDI02 and a common ground first. Without them a correct SPI implementation looks like a broken one.
- The FPGA node's single-wire bus becomes a real differential CAN bus through a transceiver, whose transmit input is to be **low for dominant**. Terminate with 120 Ω at each end.
- Climb the ladder in order, and **never have more than one unknown at a time**.
- Rung 1 (the loopback) is the transport's only test — that class is covered by no automated test.
- Rung 2 (the masking to eleven bits) proves that you are talking to the register bank and not to an echo of your own wire.
- Debug with hypotheses and measurements, not by swapping wires. Suspect your own half first, prove it before you report it, change one thing at a time.

Exercises

Exercise 13.1 The ladder

REASONING

- What does rung 2 prove that rung 1 did not?
- Why is it not enough to read back the same value you wrote?
- Which rung is the only test of `AvrSpiTransport`, and why is there no automated test for that class?

Exercise 13.2 From symptom to hypothesis

REASONING

You read STATUS and get 0x00000000 every time.

- a) List at least four possible causes.
- b) Order them by how quickly they can be ruled out.
- c) Which measurement tells the first two apart?

Exercise 13.3 Whose fault?

REASONING

Your node sends a frame, but nothing shows on the bus.

- a) Is the fault in your driver, in the FPGA or in the wiring?
- b) How do you decide that *without* guessing?
- c) Write the fault report you would send to the hardware group, with what you know so far.

Exercise 13.4 The termination

CALCULATION

You measure the resistance across the unpowered bus.

- a) What should you get with correct termination?
- b) You get 40 Ω . What is wrong?
- c) You get 120 Ω . What is wrong, and when would it show?

Bus analysis

LECTURE 14 · THE HARDWARE

The traffic seen from outside, with a tool that believes neither half.

IN THIS CHAPTER

- What a bus analyser is, and why it is a third, impartial party.
- Reading a frame in a trace: identifier, DLC, data, timestamps.
- Sending *to* your node, and watching the acknowledgement work.
- Looking for the design's documented simplifications — from the outside.

14.1 Why an analyser

Until now every observation has been made by one of the two halves. Your driver says that it sent a frame; the hardware group's testbench says that the controller transmitted one. Both can be wrong in the same way, since both are written against the same document by two people who may have misread it identically.

A **bus analyser** is a third party that believes neither of you. It is a real CAN controller on the bus, and it decodes what is actually on the wires.

That is why the lab comes last and not first: only now is there traffic to look at, and only now does the answer mean anything.

Note: The analyser also acknowledges your frames. That is worth knowing: a lone node on the bus never gets an acknowledgement, and a *complete* CAN controller would then have retransmitted for all eternity. The design in this course never reads the ACK slot back, so it notices nothing — but the difference becomes visible in the trace as soon as the analyser is connected.

14.2 Reading a trace

Set the analyser to the same bit rate as the design, and check that the termination is still $120\ \Omega$ at each end — not three resistors just because there are now three nodes.

Send a frame with a known ID and a known payload from your node, and find it:

Column	What you check
Identifier	Does it match what you wrote to TX_ID?
DLC	Does it match TX_DLC?
Data	Are the bytes right, <i>and in the right order</i> ? This is where a swap of TX_DATA_L0/HI finally becomes visible.
Timestamp	How often do they arrive? Does that match how often your code calls send()?

Then the other way round: send a frame *from* the analyser to your node. Does receive() return true? Is the content right? And — the thing that is easy to forget — did STATUS bit 1 go low when you wrote RX_ACK?

14.3 Looking for the simplifications

The design is simplified compared with real CAN in several respects. Some of them can be seen from the outside, and that is the most interesting part of the lab.

Simplification	What you can look for
No ACK read-back	Disconnect the analyser and transmit with only your node on the bus. What does the error flag say? What <i>would</i> a real node have done?
No error frames	What happens if the analyser sends one?
No automatic retransmission	A frame that went wrong is not resent by the hardware. Can you see that?
No receive queue	Let the analyser send two frames close together. Do both reach your application?

Finding a discrepancy is not the same as finding a fault. The task is to be able to say *which documented simplification* explains what you see — and which discrepancies have *no* such explanation. It is the latter that are interesting, for both classes.

14.4 Bus load

The analyser shows the bus load. Compare it with the time budget in chapter 8:

- What is the load with your node as the only transmitter?
- A send() takes six SPI transactions, that is, around 270 μs . A maximal CAN frame is around 130 μs . **What is actually limiting you — the bus or the link?**

The answer to that last question is the whole point of measuring instead of assuming, and it is the one place in the course where you can see it with your own eyes.

14.5 One frame, all the way

Finish by tracing a single frame through the whole stack, naming each layer and who built it:

```
app::EchoNode          -> you
driver::can::Interface -> you
driver::can::Spi        -> you
driver::can::ByteTransport -> you
AvrSpiTransport         -> you
===== four wires =====
spi_slave               -> supplied
spi_reg_bridge          -> the hardware group
register_bank            -> the hardware group
can_controller          -> the hardware group
===== transceiver =====
the CAN bus             -> physics
```

Ten layers, two classes, one contract — and a frame that arrived.

Summary

- A bus analyser is a **third party that believes neither half**. Until now every observation has been made by one of them, and both may have misread the contract identically.
- The analyser acknowledges your frames. A lone node never gets an acknowledgement — and this design does not even notice that, since it never reads the ACK slot back.
- In the trace a swap of TX_DATA_L0/HI finally becomes visible, because it gives a frame that arrives with the right length and the wrong order.
- Finding a discrepancy is not the same as finding a fault. The task is to connect it to a documented simplification — or to establish that it has no explanation.
- A `send()` takes longer than the frame it sends. The SPI link is the bottleneck, not the CAN bus, and here you can measure it.

Exercises

Exercise 14.1 Read your own traffic

CODE

Send a frame with a known ID and a known payload.

- Find it in the trace. Do the identifier, the DLC and the data match?
- Is the *order* of the data bytes right?
- Which fields do you see in the trace that the register map does *not* expose?

Exercise 14.2 The other way round

CODE

Send a frame from the analyser to your node.

- Does `receive()` return `true`?
- What happened to STATUS bit 1, before and after your `RX_ACK`?
- Send two frames in quick succession. Do both reach the application? Why?

Exercise 14.3 The acknowledgement

REASONING

- Does the analyser acknowledge your frames? How can you tell?
- Disconnect it and transmit with only your node on the bus. What does the error flag say?
- What would a complete CAN controller have done in that situation, and why does this one not?

Exercise 14.4 The bottleneck

CALCULATION

- What is the bus load with your node as the only transmitter?
- How long does a `send()` in your driver take, and how long does the frame take on the bus?
- What is limiting you — the bus or the SPI link? Does the measurement match the calculation in chapter 8?

Revision

LECTURE 15 · THE HARDWARE

What is examined, and how the two halves fit together.

IN THIS CHAPTER

- The two halves’ shared questions, put side by side.
- What the written test covers, part by part.
- The fifteen mistakes that have cost the most time in this project.

15.1 The same questions, two answers

The whole book revolves around five questions. The first part answered them in software, the second part let the hardware do it:

Question	Your own protocol	CAN
Where does the message begin?	SOF, two bytes	The SOF bit; seven recessive bits as an end marker
How long is it?	LEN, one byte	DLC, four bits, 0–8 data bytes
Who is it for?	DST and SRC	No address. The identifier describes the content
Did it arrive intact?	A summing checksum	CRC-15, plus bit stuffing and format rules
Did it arrive at all?	ACK, timeout, retransmission, duplicate detection	One ACK bit inside the frame

Being able to fill in that table from memory, and to justify every row, is roughly what the written test requires of the first half.

15.2 What the written test covers

15.2.1 Part A — your own protocol

Frame design and checksum calculation by hand. Parsing as a state machine. Addressing, routing and sequence numbers. Byte streams to be interpreted and analysed. The error model: dropped bytes, corrupt data, delay. The chain ACK → timeout → retransmission → duplicates.

15.2.2 Part B — CAN and the driver

CAN IDs, priority, DLC and arbitration. Which mechanisms the hardware performs, and where. The register map: what every register means, and how every status bit is cleared. The data byte order across TX_DATA_L0/HI. The SPI transaction: the command byte, the four data bytes, the SS rules, and encoding and decoding a transaction by hand. The layered architecture. The three limitations.

The register map is handed out with the test — the indices do not have to be memorised. *Reading* the map and knowing what it means is all the more necessary.

15.3 The most expensive mistakes

Each of them has cost somebody an evening. They are listed in the order they tend to happen.

#	Mistake	Symptom
1	The parser does not check the length field against the size of the buffer	Buffer overflow; all sorts of other things break
2	The parser goes back to <code>WaitForSof1</code> on a second <code>0xA5</code>	The odd frame is missed, inexplicably
3	Multi-byte fields are serialised by copying memory	Works until the other side is built with a different endianness
4	A duplicate is not acknowledged again	The sender retransmits for all eternity
5	<code>TX_DATA_LO</code> and <code>TX_DATA_HI</code> swapped	The frame arrives — back to front
6	The cast after the shift in <code>readReg()</code>	Works for small values, breaks at bit 7
7	<code>TX_SEND</code> written before the data registers	The node runs exactly one frame behind
8	<code>clearError()</code> writes <code>0x1</code>	The error bit never goes low; all error handling stops working
9	<code>RX_ACK</code> forgotten	The same frame is read in an endless loop
10	<code>receive()</code> does not mask against <code>dlc</code>	Up to six bytes are reported that were never sent
11	<code>while (!send(frame)) {}</code> on an invalid frame	Hangs forever
12	The pin mux not set	The program runs, the wires are silent, no errors are reported
13	The data register is not read in <code>transfer()</code>	All the data is one byte out
14	<code>VDDIO2</code> not connected	Looks exactly like a broken SPI implementation
15	The factory's members in the wrong declaration order	A reference to an object that has not been constructed

Note: Eleven of the fifteen would have been caught by a test on the laptop. Three of them — 12, 13 and 14 — would not, and those are precisely the three that live in the one class no automated test covers. That is no coincidence, and it is the whole argument for the ladder in chapter 13.

15.4 After the course

Two things continue directly from here.

Testing. Throughout the project you have run and read tests somebody else wrote. The next step is deciding what they should check — which is a harder problem than it sounds, and the one that decides whether a test suite is worth anything.

Your own repository. The driver you have built is a complete, tested, layered driver stack against real hardware, developed against a contract and verified without access to the other half. That is a reasonable first item in a portfolio, and a more realistic description of embedded development than most course projects.

Solutions

Here are the solutions to **every exercise that does not ask for code** — all the reasoning, calculation and decoding exercises, and the parts of the coding exercises that are reasoning. The solutions are complete: where an exercise asks for a calculation the steps are there, and where it asks for reasoning the argument is there and not just the conclusion.

The exercises that ask for code are not answered here. For those, the supplied test suite is the answer: it says, test case by test case, exactly what the code is to do. The code is also published in the course repository after the lecture in question.

Read the solution after you have tried yourself. A solution you have read is not the same thing as a solution you have arrived at, and that difference is what the exercises exist for.

Chapter 1 — Frames

Exercise 1.1 Build a frame

page 5

a) $LEN = 0$, $TYPE = 0x02$, $DST = 0x25$, $SRC = 0x17$, $SEQ = 0x7F05$. The checksum becomes $A5 + F7 + 00 + 02 + 25 + 17 + 7F + 05 = 606 = 0x025E$:

```
A5 F7 00 02 25 17 7F 05 02 5E
```

b) The addresses swap places, $LEN = 1$, $TYPE = 0x03$, one data byte $0x16$. The checksum becomes $630 = 0x0276$:

```
A5 F7 01 03 17 25 7F 05 16 76
```

c) Ten bytes: the eight header bytes plus two checksum bytes, with $LEN = 0$.

Exercise 1.2 The fields and their purpose

page 5

Without **SOF** there is no way to know where a frame begins in the stream. Without **LEN** the receiver does not know when the payload ends, since it reads one byte at a time. Without **TYPE** there is no way to tell what the frame means. Without **DST** nobody knows who it is for, without **SRC** the receiver cannot answer, and without **SEQ** replies cannot be paired with requests and duplicates cannot be recognised. Without **CHK** no transmission errors are detected.

The two that can be removed in a system with exactly two nodes are **DST** and **SRC**: with only two parties, the sender and the receiver are given by who received it. That is also precisely what makes them indispensable as soon as a third node is added.

Exercise 1.3 Endianness

page 5

- a) 05 7F — x86 stores little-endian, that is, the lowest byte first, while the protocol requires 7F 05.
- b) Both sides have the same bug. The serialisation writes 05 7F and the deserialisation reads it back as 0x7F05, so the errors cancel each other out as long as both ends are built from the same code on the same architecture.
- c) As soon as the other side is something else: a different architecture, a different implementation, or a bus analyser reading the stream according to the specification. Then 0x7F05 becomes 0x057F.

Exercise 1.4 The limits of the checksum

page 6

- a) No. A sum is independent of the order, so two swapped bytes give the same sum.
- b) No. The sum is unchanged, since what was added in one place was subtracted in another.
- c) For example: swap **DST** and **SRC** in the frame from exercise 1.1, or change 0x25 to 0x26 and 0x17 to 0x16. Both give an identical checksum. A CRC would detect either, which is why CAN uses one.

Chapter 2 — Byte parsing

Exercise 2.1 Find the frame

page 11

- a) At index 4 (zero-indexed): 32 74 00 FF is junk, and A5 F7 at index 4–5 is the SOF.
- b) A5 F7 03 03 25 17 7F 05 10 20 30 02 C2 — thirteen bytes, that is, ten header and footer bytes plus three data bytes.
- c) Discard them. They are received in `WaitForSof1` and rejected one by one, since none of them is 0xA5.
- d) `TYPE = 0x03`, that is, `StatusResponse`, from `SRC = 0x17` to `DST = 0x25`.

Exercise 2.2 The SOF trap

page 11

- a) The naive implementation goes back to `WaitForSof1` and discards the byte. It thereby discards the byte that may just have been the new frame's first.
- b) Stay in `WaitForSof2`. The byte was 0xA5, that is, a possible start of SOF in its own right.
- c) For example A5 A5 F7 ...: here the second 0xA5 is the frame's real beginning. The naive parser discards it, ends up in `WaitForSof1`, sees 0xF7 — which is not 0xA5 — and misses the whole frame.

Exercise 2.3 A dropped byte

page 11

- a) `LEN` is still 3, so the parser collects 10 30 02 as the payload and then reads C2 as the *first* checksum byte.
- b) Not on this stream. The parser is in `WaitForChk2` waiting for a byte that never comes; only when the next byte arrives is the framing complete.
- c) `false`. The framing can succeed, but `Frame::deserialize()` recomputes the checksum and it does not match.
- d) `WaitForChk2` until the next byte arrives. To find the next frame the caller has to call `reset()`, after which the parser looks for 0xA5 again.

Exercise 2.4 The length field

page 11

- a) Set LEN to a value greater than `MaxPayloadLen`, for example `A5 F7 FF 00 ...`. The parser then collects 255 data bytes into a buffer that holds fewer.
- b) Too late. The damage is done during the collection, that is, in `processByte()`, long before anybody gets to call `extractFrame()`.
- c) That every length coming from outside is to be checked against the size of the buffer *before* it is used. The same rule applies to TX_DLC in chapter 10: the hardware does not validate, so the driver has to reject `dLc > 8` before it writes anything.

Chapter 3 — Buses, routing and reliability**Exercise 3.1** Routing

page 17

- a) In the node. A physical wire delivers voltage levels; it can neither read addresses nor know that frames exist. Every node therefore has to compare DST with its own address itself.
- b) Ignore it. The frame is not faulty — it is addressed to somebody else.
- c) Because Nack means *something went wrong*. On a bus with n nodes every frame would trigger $n - 2$ false negative acknowledgements, and the sender would not be able to tell them from a real error.

Exercise 3.2 The error model

page 18

- a) **Dropped byte:** no — every following field is shifted, and LEN is read from the wrong byte. **Flipped bit:** yes — the structure is intact, only the content is wrong. **Delay:** yes — nothing in the frame is affected.
- b) The first two are caught by the checksum, in `deserialize()`. The third should be caught by nothing, since nothing is wrong.
- c) After a dropped byte or a flipped bit the parser is left with a rejected frame and has to be reset; after a delay it is where it should be.

So the delay should cause no problem. If it does anyway, the parser has a hidden timing dependency — it assumes something about *when* bytes arrive — and that is a bug which does not show until the system is moved to slower hardware.

Exercise 3.3 A lost acknowledgement

page 18

- a) A gets no Ack, the timer expires, and A retransmits the frame with the **same** SEQ.
- b) B sees a frame it has already processed: the same sender, the same sequence number.
- c) B is to **send Ack again** and **not** run the application logic a second time. The command has already been carried out.
- d) A would have timed out again and retransmitted again, in a cycle broken only by the attempt counter running out. That is why the missing acknowledgement is the dangerous error, not the missing frame.

Exercise 3.4 The timeout value

page 18

- a) Unnecessary retransmissions and thereby more duplicates, plus increased bus load precisely when the bus is already loaded.
- b) Slow error detection: the system stands waiting for an answer that never comes, and feels hung.
- c) The worst-case time for a round trip: the transmission time, the receiver's processing time and the acknowledgement's way back. A node rarely knows it, since it depends on the whole system's load and on how fast the other node is — which is exactly what varies.

Exercise 3.5 The chain

page 18

The checksum detects that a frame is broken, but says nothing about frames that never arrived. **The ACK** confirms delivery, but does not help when the acknowledgement itself fails to arrive. **The timeout** detects the missing acknowledgement, but does nothing about it. **Retransmission** fixes it, and thereby creates duplicates. **Duplicate detection** cleans up after the retransmission, and creates no new problem — which is why the chain ends there.

Chapter 4 — CAN, the register map and the architecture

Exercise 4.1 Who does what

page 25

Frame synchronisation, the length field, the checksum, the acknowledgement and collision handling are all done by **the hardware**: SOF and EOF, the DLC, CRC-15, the ACK bit inside the frame, and the arbitration the bus resolves electrically.

The two rows without an obvious answer are **addressing** and **sequence numbers**. CAN has no destination address and no sequence number at all: the identifier describes the content, and if a sequence number is needed the application has to put it in the payload.

Timeout and retransmission together with **duplicate detection** are a third case: a complete CAN controller handles the retransmission in hardware, but the simplified one in this design does not — see §4.3.4.

Exercise 4.2 Pack a frame

page 25

Data byte 0 sits in the most significant bits, and TX_DATA_L0 carries the *first* four bytes:

```
TX_ID      = 0x000002A7
TX_DLC     = 0x00000005
TX_DATA_L0 = 0x01020304
TX_DATA_HI = 0x05000000
TX_SEND    = 0x00000001
```

The order matters because the write to TX_SEND does not store a value but *triggers a transmission*: the controller then reads the other registers. If the trigger is written first, the previous frame's contents are sent.

Exercise 4.3 Unpack a frame

page 25

- a) 0x481.
- b) DE AD BE — three bytes, since RX_DLC is 3 and RX_DATA_L0 carries bytes 0–3 with byte 0 at the top.
- c) Zero. The driver is to mask against dlc; bytes above the received length are not the frame's content, even if the hardware happens to zero them.
- d) RX_ACK, with 0x1. Bit 0 has to be set.

Exercise 4.4 Four statements

page 25

a, b and c describe bugs.

- a) Bit 0 means *the port is free again*, not *the frame arrived* — a lost arbitration gives the same bit back. Read `ERROR_FLAGS` afterwards.
- b) `ERROR_FLAGS` is cleared only by an *all-zero* word. `0x1` does nothing, so the error bit never goes low.
- c) Bytes above `RX_DLC` do not belong to the frame. Copying all eight reports up to six bytes that were never sent.
- d) Correct. Without the check, a write can happen in the middle of an ongoing transmission.

Exercise 4.5 The layer boundary

page 26

- a) False. `EchoNode` sits above the interface and must not know about any concrete driver.
- b) True. `Spi` is an `Interface` and has to include it.
- c) False. The stub simulates CAN in memory and has no transport.
- d) False. The transport moves bytes and knows nothing about registers — that is the whole point of it being dumb.
- e) False. `EchoNode` is tested through `Stub`, which has no transport.

Chapter 5 — The frame in code, and the first test suite**Exercise 5.1 Predict**

page 31

- a) Typically 12 bytes, not 11. The compiler inserts padding so that `id` (`std::uint16_t`) lands on an even address, and ends the struct at a size that is a multiple of its strictest alignment.
- b) Yes. `f.data[7]` is 0: aggregate initialisation zeroes the members that are not given.
- c) The line stops compiling. A user-declared constructor makes the type a non-aggregate, and then there is no constructor taking `(0x1230, 20, {...})`. That is precisely why `Frame` has none.

Exercise 5.2 `constexpr`

page 31

- a) That `isValid()` can be evaluated at compile time — otherwise it is not a valid `static_assert` condition. That also proves that the function is free of side effects.
- b) Nothing. The check happens when the program is built; in the finished binary there is no code for it at all.

Exercise 5.3 Read the suite

page 32

- a) `0x7FF` and `0x800` for `id`, 8 and 9 for `d1c` — the largest that is to be accepted and the smallest that is to be rejected. Bugs live on the boundaries: a value in the middle of the range passes both a correct and an incorrect comparison.
- b) The case checking that `d1c = 8` is accepted would fail; all the others would be green. That is why the pair is needed and not just one of the values.
- c) Yes — that `MaxDataLen` is the same number as the size of the array is not checked, and cannot be checked from outside. A green suite means that the code is right on the points the suite asks about, not that it is right.

Exercise 5.4 Break the code

page 32

- a) The case accepting the largest valid `id`, that is, `0x7FF`. The output names the expression, the file and the line.
- b) Shorter than guessing: the test name points out the requirement, the Arrange part the conditions, and the Assert line the expected value — only after that do you need to open your own code.
- c) Restore the code — and note that the incorrect version passed every other test case. A single character separated a correct implementation from one that rejects every eighth valid frame.

Exercise 5.5 Where the validation sits

page 32

- a) That an error is caught earlier and closer to whoever caused it.
- b) That the rule now exists in two places, and that both have to be kept in step.
- c) Only one of the places is changed. The forgotten copy then rejects frames that are valid, or lets through frames that are not — and the fault looks as though it comes from the hardware. The rule is expressed once, in `isValid()`, and used in one place: in `Spi::send()`.

Chapter 6 — The interface and the stub

Exercise 6.1 Predict the compiler

page 36

- a) It compiles without the slightest objection, and that is the whole problem. Without `virtual`, which destructor is called is decided by the pointer's *static* type, that is, `Interface`. `Stub`'s destructor never runs, its members are never destroyed, and the standard calls that undefined behaviour. For `Stub` it is not noticed — it owns nothing — but for a class holding a resource it leaks, silently, on every deletion. No test catches it.
- b) Without the attribute, `driver.send(frame);` is a perfectly legal statement the compiler has no reason to complain about. With `[[nodiscard]]` it becomes a warning, and with `-Werror` an error. That turns the most common driver bug — believing that a frame went out when the hardware was busy — from a debugging session into a compilation that stops.
- c) With `override` the misspelling is a compile error: there is no virtual method by that name to override. Without `override` it compiles, and you have instead *added* a new method. The base class's pure virtual method is then still unimplemented, so the class remains abstract — and the error message you get is about the type not being instantiable, far from the misspelled line. That is why `override` is mandatory.

Exercise 6.2 One method too many

page 36

- a) Nothing meaningful. The stub has no registers; it would have to return an invented constant, that is, to lie.
- b) `EchoNode` becomes tied to the hardware *having* registers. The test for whether a method belongs in the interface is exactly this: can both implementations fulfil it honestly? If they cannot, it does not belong there.
- c) Every caller has to change, since the meaning of the bits is part of the register map and not of the interface. The interface would then inherit the hardware's versions, which is precisely what it exists to avoid.
- d) In `Spi`, as a private helper function — or, for debugging, in a separate `PrintTransport` below the `ByteTransport` seam, where it is invisible to anybody above.

Exercise 6.3 The polymorphism

page 37

The exercise does not ask for code; the questions are answered here.

- a) No. The function takes a `driver::can::Interface&` and uses only the interface's four methods.
- b) Because it can then be run against any implementation at all. Written against `Stub` it would have been a test of `Stub`; written against the interface it is a test of the *contract*, and the contract is what both `Stub` and `Spi` promise.
- c) That `Spi` really does fulfil the same contract: the same return values in the same situations. Above all, `send()` has to return `false` when the hardware is busy and `receive()` has to return `false` when there is no frame — otherwise the function passes for the stub and fails for the driver.

Exercise 6.4 The limits of the stub

page 37

- a) Errors (`hasError()` is always `false`), busy hardware (`send()` always succeeds), a queue (a second `send()` overwrites the first), and everything CAN does: arbitration, CRC, bit stuffing.
- b) Busy hardware, for example: a counter that makes the next `send()` return `false`, set by a test method. That is the simplest, since it requires no new state beyond a flag.
- c) Every simulated behaviour is code that can be wrong. A stub that simulates a queue, errors and timing is a second implementation of the hardware — and then you no longer know whether a failing test is due to the driver or to the stub. The limit lies roughly where the stub would need a test case of its own: if it does, the behaviour belongs in a more realistic double, such as `BankTransport` in chapter 11.

Chapter 7 — Component testing through the stub**Exercise 7.1 Write the bug, watch the test catch it**

page 43

The exercise asks for a code change; the questions are answered here.

- a) The cases checking the echoed frame's content and the counter fail — the frame that is echoed has the wrong `d1c`, and the case calling `run()` twice on one injected frame reveals that the state has been consumed. The case without an injected frame is still green, since `receive()` returns `false` already the first time.
- b) No, neither of them. `Stub` and `Frame` do exactly what they promise; neither of them is broken.

c) A unit test cannot see a bug that lives *between* the units. It takes a test running several parts together to discover that one of them uses the other incorrectly.

Exercise 7.2 The layer boundary

page 43

- a) No. EchoNode has to know the interface — it is its only dependency.
- b) No. The test holds the concrete type; that is where `inject()` lives.
- c) Yes. The implementation file is also above the interface, and must not know which driver it is running against.
- d) No, it does not break the boundary — `hasError()` and `clearError()` belong to the interface. But it is questionable anyway: EchoNode cannot do anything meaningful about an error it cannot interpret, and the error flag is one bit for four causes. Error handling belongs with whoever owns the driver and knows the system's requirements, that is, in the factory or in the layer above.
- e) Yes, grossly. The cast names a concrete driver *and* reaches a private method in it. It would also crash as soon as a Stub was injected.

Exercise 7.3 Ownership

page 43

- a) It does not compile. Interface is abstract and cannot be instantiated as a value; the compiler says that the type is abstract. Were it not, the member would *slice* the object — copy the base class part and throw the rest away — which is worse than a compile error.
- b) It compiles. The reference then points at a destroyed object, and `run()` is undefined behaviour. The compiler cannot see it; that is the one cost of the component not owning its driver.
- c) Because EchoNode does not own it. The driver's lifetime is longer than the component's, it can be shared with other components, and the ownership sits with the factory. A `unique_ptr` would have claimed the opposite, and a `shared_ptr` would have added counting that nobody needs.

Exercise 7.4 Read the suite

page 44

- a) The case checking only the echoed frame's *content* — the right `id`, `dlc` and `data` — would be green, since the echo still happens. That is why the suite also checks the counter.
- b) Traffic is fed in with `stub.inject()` and read back with `stub.receive()`. Neither of them is in Interface: `inject()` Spi cannot implement, and `receive()` is used here in a different role from the interface's — the test inspects what the component *sent*, which the stub happens to store in the same field.

c) That `run()` consumes the frame: after the first call there is nothing left to echo, so the counter is to stand at 1 and not 2. That is the requirement catching the double call in exercise 7.1.

Chapter 8 — SPI and the transaction protocol

Exercise 8.1 Decode

page 48

The command byte is `W` in bit 7, zeroes in bits 6–4, and the register index in bits 3–0.

- a) `0x81` = write bit set, index 1: a **write to TX_ID** with the value `0x000007FF` — eleven bits of ones, that is, the largest value the register can hold.
- b) `0x85` = write, index 5: a **write to TX_SEND** with `0x1`, which *triggers a transmission*. The register stores nothing.
- c) `0x8B` = write, index 11: a **write to ERROR_FLAGS** with `0x0`, which clears the error bit.
- d) `0xC2` is `1100 0010`: the write bit set, but *also* bit 6, which is reserved and has to be zero. The command is invalid — no write is committed, no read is latched — and the slave consumes all five bytes anyway before returning to idle.

The two unusual ones are **c** and **d**. In **c** you clear by writing *zero*, which is the opposite rule to `TX_SEND` and `RX_ACK`, which require bit 0 to be set. In **d** the command is invalid and the slave does *not* report it: there is no error channel at this layer, so a driver sending junk gets silent nonsense back.

Exercise 8.2 Encode

page 48

- a) A read has bit 7 low, so the command byte is just the index; `RX_DLC` has index 7 and the four data bytes are dummy bytes: `07 00 00 00 00`
- b) `82 00 00 00 04`
- c) `85 00 00 00 01`
- d) `8A 00 00 00 01`
- e) `8B 00 00 00 00`

The difference between **d** and **e** is the rule for what counts as a valid write. `RX_ACK` requires **bit 0 to be set** — a zero does nothing. `ERROR_FLAGS` requires **the whole word to be zero** — `0x1` does nothing. Two registers next to each other in the same map, with opposite rules, and confusing them is one of the most common bugs in the project.

Exercise 8.3 What comes back

page 49

- a) None. During a write, everything the slave drives on MISO is meaningless.
- b) Four: the last four. The byte clocked in *during the command byte* is leftovers from last time, since the slave does not load its response shifter until the end of the command byte.
- c) The return value is shifted by one byte: the first real data byte ends up in bits 23–16 instead of 31–24, and the last data byte falls off entirely. While debugging it looks as though every register value is 256 times too small, or — for STATUS — as though the bit you are looking for is never set. That is the same symptom as when `SPI0.DATA` is not read in `transfer()`, which makes the two easy to confuse.

Exercise 8.4 SS and aborts

page 49

- a) The transaction is abandoned entirely, without side effects: `TX_ID` is not changed, no trigger fires, and the slave returns to idle ready for the next falling SS edge. That is what makes the slave self-healing after a glitch.
- b) The slave takes the first five bytes as a transaction and **discards the following five**. A transaction only begins on the first byte after SS has fallen, so the sixth byte is never interpreted as a new command byte.

The dangerous part is that it fails *silently*. At bring-up it looks as though every other register write disappears: the first arrives, the second does not, and nothing reports an error. You suspect the wiring, the clock frequency and the driver's bit packing — long before you suspect SS.

- c) `0x91` is `1001 0001`: the write bit set, index 1, but bit 4 is set and reserved. The command is invalid and is ignored entirely. It should be prevented already when the command byte is built: `CmdWrite | index` with an index in the range 0–12 can never set bits 6–4, which is why the index is a named constant and not a number from a calculation.

Exercise 8.5 Time budget

page 49

- a) Six transactions — `STATUS`, `RX_ID`, `RX_DLC`, two data registers and `RX_ACK` — that is, around $6 \times 45 = 270 \mu\text{s}$.
- b) A maximal frame is around 130 bits, that is, about **130 μs** at 1 Mbit/s.
- c) They overwrite the previous one. The controller holds exactly one received frame, and the newest wins.

That is **not** your bug: it is the hardware's documented limitation, and it is deliberate. You can reduce the risk by polling often and not doing anything slow between the calls, but you cannot get past the fact that a `receive()` takes twice as long as the frame it fetches. At sufficient traffic intensity frames are lost whatever the driver does.

Chapter 9 — The transport seam and the register access

Exercise 9.1 Predict

page 56

- a) RegTxDataHi has index 4. Write: `CmdWrite | 4 = 0x84`. Read: just the index, that is, `0x04`.
- b) `83 AA BB 00 00` — the command byte `CmdWrite | 3`, then the value MSB first.
- c) `0x12345678`. The first byte, `0xFF`, is the answer to the command byte and is discarded; the four following ones are assembled.
- d) You get `0xFF123456`: the value is shifted by one byte, the last data byte falls off, and the junk byte ends up at the top. The symptom is register values that look 256 times too large with an inexplicable `0xFF` at the top.

Exercise 9.2 The sign extension

page 56

The exercise asks for a code change; the questions are answered here.

- a) value becomes `0xFFFFFFFF80000000` truncated to 32 bits, that is, `0x80000000` — but the route there goes through undefined behaviour. `transfer()` returns a `std::uint8_t`, which is promoted to `int`. `0x80` shifted 24 steps to the left sets the sign bit in a 32-bit `int`, and shifting into the sign bit is undefined in C++17. In practice the result is a negative number that is then converted to `std::uint32_t` with ones where there should be none.
- b) For every byte where bit 7 is *low*, that is, all values below `0x80`. The code therefore works perfectly for small identifiers and small DLC values.
- c) The driver would have worked for `id` up to `0x7F` in the topmost data byte and broken above that. Concretely: `0x123` works, `0xAABB` in the data field does not. At bring-up it looks as though some frames arrive and others do not, depending on the *content* — which is one of the most confusing symptoms there is, since you go looking for a fault in the transfer when the fault is in a type conversion.

Exercise 9.3 A transport to debug with

page 56

The exercise asks for code. The output should be comparable line by line against the example in chapter 8: five bytes between a `select` and a `deselect`, the command byte first, the value MSB first.

Two things are worth building in from the start: print *both* the sent and the received byte on every line, so that a read can be followed, and count the transactions, so that a `send()` can be checked against the six it should cost.

Exercise 9.4 The layer boundary

page 57

- a) No. `ByteTransport` moves bytes and knows nothing about frames. A dependency on `frame.h` would be the first step towards it knowing what it transports.
- b) No. Then it would no longer be a transport but a register access, and that belongs in `Spi` — the only layer that is to know the register map.
- c) No, and this is the one that sounds the most reasonable. The argument for is that the transaction is always five bytes anyway, so why not let the transport handle it? The argument against is that the SS framing and the five-byte format belong to the *protocol*, not to the transfer. Put them in the transport and every transport implementation — including the scripted test double — has to know the protocol, and then there is no seam left: you can no longer swap out how bytes are moved without swapping out what they mean at the same time.
- d) No. Testing a private method directly is almost always a sign that it ought to be public, or that the test is testing the wrong thing. Here it is tested through `send()` and `receive()`, and by inspecting which bytes came out — which is precisely what has to be true.

Chapter 10 — The driver

Exercise 10.1 Break the code on purpose

page 64

The exercise asks for code changes; the outcomes are described here.

- a) The trigger is written before the data registers, so the controller reads the previous frame's contents. On the bus you see a node running exactly one frame behind — it always sends what you asked for *last* time.
- b) Data bytes 4–7 end up in `TX_DATA_L0`. A frame with `d1c = 2` and data `AA BB` is sent as two zero bytes; the receiver gets the right length and the wrong content.
- c) `AA BB CC DD` arrives as `DD CC BB AA`. The frame arrives, with the right length — just back to front.
- d) The error bit never goes low, so `hasError()` returns `true` forever after the first error and all error handling above stops working.
- e) `STATUS` bit 1 stays set. Every `receive()` returns `true` with the same frame, in an endless loop, and no new frame can be received.
- f) Nothing shows in the output, and that is the point. `PrintTransport` shows which bytes went out, and they are correct — the fault is in what the driver *hands back* to the caller, that is, in `frame.data` above `d1c`. It is caught by a test case delivering a frame with `d1c = 2` after one with `d1c = 8` and checking that the six topmost bytes are zeroed, which requires a double

modelling the hardware's behaviour — that is, `BankTransport` in chapter 11, not the scripted one.

g) Writes happen in the middle of an ongoing transmission and overwrite data the controller is using. The outcome depends on the timing, which makes the fault sporadic.

Exercise 10.2 Count transactions

page 65

- a) Six: one `STATUS` read and five writes.
- b) One — the `STATUS` read. After that it returns `false`.
- c) **Zero.** The validation happens before everything else, so an invalid frame costs not a single transaction.
- d) One.
- e) Six: `STATUS`, `RX_ID`, `RX_DLC`, two data registers and `RX_ACK`.

Question c is the point of the order: a caller that happens to send invalid frames in a loop does not load the bus at all.

Exercise 10.3 The infinite loop

page 65

The exercise asks for code; the outcomes are described here.

- a) The loop spins until the hardware becomes free, and then ends normally. That is the intended behaviour.
- b) The loop spins forever. `isValid()` returns `false` however long you wait, and nothing in the system is going to change that.
- c) Validate before the loop and return immediately on an invalid frame, and count attempts in the loop so that even hardware that never becomes free gives up. See §10.5.

Exercise 10.4 Design questions

page 65

- a) An enumeration as the return value instead of `bool`, for example: `Accepted`, `Busy`, `Invalid`. The cost for `Stub` is small — it would return `Accepted` or `Invalid` — but the cost for *the interface* is that it grows, and every caller has to handle more cases. The trade-off is that a `bool` is simpler to implement and harder to use correctly.
- b) A test case delivering a frame with `dLc = 8` and then one with `dLc = 2`. Without masking, the second frame still carries six bytes from the first, and the application cannot tell them from real data. On real hardware it also takes no more than the hardware group changing when the data accumulator is cleared, or you borrowing a board built by another group.
- c) The driver observes that `STATUS` bit 0 goes low at the trigger and comes back shortly afterwards — exactly as on a successful transmission — and that the error flag is set. It

cannot observe that it was an arbitration that was lost rather than a stuff error, a bad CRC or a received remote frame, and it cannot see which node won or which frame went through instead.

d) Polling more often, that is, doing less work between the `receive()` calls. The limit you cannot get past is that a `receive()` takes around 270 μs while a frame takes 130 μs : a transmitter running continuously always has time to send more frames than you have time to fetch.

Chapter 11 — Testing without hardware

Exercise 11.1 Get the suite green

page 71

The exercise asks for code. The log, on the other hand, is the point: write down what the test name said, where the bug was, and how long it took.

What you usually see is that the time falls sharply as the exercise goes on, and the reason is not that the bugs get easier but that you stop guessing. The procedure in §11.5 — test name, index, expected value, and *only then* your own code — is the whole difference.

Exercise 11.2 Which double catches what?

page 71

a) **Scripted.** The recorded bytes end up in the wrong transaction, which shows immediately. `BankTransport` obediently writes the values into the registers the command byte points out and notices nothing.

b) **Both.** Scripted sees that the sent byte is 01 where the test expects 00; Bank sees that the error bit never goes low.

c) **Bank only.** From the scripted transport's point of view exactly the right bytes were sent — the fault is in what the driver hands back in `frame.data`, not in what it sent.

d) **Both.** Scripted is missing the `RX_ACK` transaction in the recording; Bank sees that `RX valid` stays set and that the next `receive()` gives the same frame.

e) **Bank only**, indirectly. Scripted checks what was *sent*, and that byte is sent correctly — the fault is that the answer is assembled wrongly. Bank gives back values that do not match what was written, so some read comes out wrong.

f) **Both.** Scripted sees that transactions go out even though the frame is invalid; Bank sees that the registers are written.

g) **Neither.** And that is precisely why the row is there: a wiring fault is invisible to every test run on the laptop, however good the test double is. That is what chapter 13 exists for, and the reason rung 1 of the bring-up ladder is a loopback.

Exercise 11.3 Read the suite as a requirements specification page 72

- a) The test case checking the TX_DATA_L0 transaction on a `send()` of `id = 0x123`, `d1c = 2`, data AA BB. Its five `EXPECT_EQ` lines are the example's five bytes, written as a requirement.
- b) The case sending a frame with `d1c = 9` and checking that the recorded byte stream is *empty*, that is, that no transactions went out at all. It checks not only the return value but that the validation happens *first*.
- c) The distribution should lean towards `BankTransport` for behaviour and `ScriptedTransport` for the protocol — roughly as in the table in §11.1. If you see only one of them in the suite, a whole class of bugs is unaccounted for.

Exercise 11.4 The whole stack page 72

The exercise asks you to run a test case; the questions are answered here.

- a) Zero. `EchoNode` is unchanged since chapter 7.
- b) That `EchoNode` depends on `driver::can::Interface` and nothing else. Both `Stub` and `Spi` fulfil that contract, so the swap is a single line in the test's `Arrange` part.
- c) Everything. `EchoNode` would have had to include `spi.h`, and thereby dragged in `ByteTransport` and the whole transport layer. The test would have needed a transport just to construct the component, and the component test in chapter 7 — which runs entirely without a transport — could not have been written.

Exercise 11.5 What remains page 72

At least these, and the last one is the one usually forgotten:

- That the **interpretation** of the contract is right. The suite is written by a human who read the same document as you.
- That the **hardware group** interpreted it the same way. Their testbenches are written against the same document, by somebody else.
- That the **wiring** is right — a swapped `MOSI/MISO` is invisible here.
- That the **time budget** holds. `BankTransport` answers immediately.
- That you **poll often enough** not to lose frames.
- That `AvrSpiTransport` works. It is the only class in the project that no automated test covers, since it talks to a physical register.

Chapter 12 — The hardware, the transport and the factory

Exercise 12.1 Watch the optimiser work

page 79

The exercise asks for code; the outcomes are described here.

- a) The loop disappears, or becomes an unconditional jump to itself. The compiler reads flag once, sees that nothing in the loop changes it, and concludes that the condition is constant.
- b) The read moves back into the loop: one load from memory per iteration.
- c) The status flag is changed by the SPI peripheral, not by the program. Without `volatile` the compiler has the same right to read it once and assume that it stands still — and then `transfer()` either waits forever or does not wait at all.

Exercise 12.2 Predict, change, observe

page 79

The exercise asks for code changes; the symptoms are described here.

- a) Nothing happens on the connected wires. The peripheral sits on the default mux, that is, a different port, and the program runs on without reporting anything. The most confusing symptom there is: no error code, no clock pulses, no clue.
- b) The master leaves master mode on the first `select()`, since it interprets the low SS level — which your own code has just created — as another master having taken over the bus. The transfer stops immediately.
- c) The clock ends up above the protocol's ceiling of 1 MHz. It may very well work on a short cable, and that is the dangerous part: the FPGA slave guarantees nothing above it, so the faults come sporadically and get worse with longer wires.
- d) The status flag is never cleared. The next `transfer()` sees a flag that is still set, returns immediately, and reads a byte that has not had time to arrive — so all the data is **one byte behind** through the whole transaction. It *looks* like it works: bytes arrive, the count is right, only the content is shifted, and it is easy to suspect the bit order instead.
- e) The clock pin is not driven, so the slave sees no edges and nothing is transferred. Unlike a), MOSI and SS are active, which means a logic analyser shows activity on three of the four wires — and points straight at it.

Exercise 12.3 The architecture test

page 80

The exercise asks you to compile; the questions are answered here.

- a) It should work. If it does not, the architecture is broken.
- b) The compiler error names the file, and that file is by definition above `driver/can/interface.h` and has learnt something about the hardware — typically an `#include` of `spi.h` or a reference to `ByteTransport`.

c) Exactly the same thing as before, against an array in memory: it echoes injected frames and increments its counter. That the behaviour is unchanged is the whole point — the application cannot tell an FPGA from a stub.

Exercise 12.4 Freestanding

page 80

The exercise asks you to build; the questions are answered here.

- a) Yes. Nothing in the project uses exceptions or RTTI.
- b) Nothing should be found. The whole stack — `Frame`, `Interface`, `Stub`, `ByteTransport`, `Spi`, `EchoNode` — is built on statically allocated objects of known size.
- c) Because the coding conventions were written for a freestanding target from the start: return codes instead of exceptions, aggregates instead of dynamic containers, virtual functions instead of downcasting. That was not luck, and this is the chapter that shows what the habit was worth.

Exercise 12.5 The construction order

page 80

- a) It compiles, and the compiler does not warn. Members are constructed in *declaration order*, whatever order the initialiser list happens to have.
- b) `myDriver` is constructed first and binds a reference to `myTransport`, which has not been constructed yet. The binding itself is harmless — it is only an address — but the first call to `transfer()` uses an object that has not been initialised, that is, undefined behaviour. On a freestanding target it usually manifests as the SPI peripheral never being configured.
- c) That a member depending on another has to be declared *after* it. The initialiser list does not express the order; it just follows the declaration order, and a compiler that warns about it only does so when the list and the declarations are in different orders — not here.

Chapter 13 — Bring-up

Exercise 13.1 The ladder

page 84

- a) Rung 1 proves that *your* SPI master works: the clock, the bit order, the timing and the configuration. The FPGA is not connected and cannot affect the outcome. Rung 2 proves that the whole chain works — the wiring, the slave's synchronisation, the transaction bridge and the register bank — and that both sides interpret the command byte the same way.
- b) Because a loopback also gives back what you wrote. If you write `0x00000123` and read `0x00000123`, that may just as well be your own MOSI coming back on MISO through a miswiring.

The masking is the proof: write 0xFFFFFFFF to TX_ID and read back 0x000007FF. An echo cannot mask away the upper bits; only a register bank can.

c) Rung 1. AvrSpiTransport is the only class in the project that no automated test covers, since it talks to a physical register — it cannot be run on the host computer, and a test double of it would have tested the double.

Exercise 13.2 From symptom to hypothesis

page 85

a) For example: MISO not connected or connected to the wrong pin; the pin mux not set, so the peripheral sits on the wrong port; VDDIO2 not powered, so PORTC does not work; the FPGA not programmed; the clock too fast; SS not released between transactions; or a missing common ground.

b) Fastest first: VDDIO2 (read the MVIO status in the program — no measurement at all), the pin mux (read the code), the FPGA being programmed (look at the board), then the measurements: clock pulses on SCK, the SS behaviour, and last the wiring, wire by wire.

c) A loopback: connect MOSI straight to MISO and run rung 1 again. If you get back what you sent, your master works and the fault is in the wiring to the FPGA or in the FPGA. If you do not, the fault is on your side, and the FPGA is ruled out.

Exercise 13.3 Whose fault?

page 85

a) It cannot be decided from the symptom alone — all three are possible, which is the whole difficulty of debugging a system two groups built one half each of.

b) Climb down the ladder until something gives the right receipt. If rungs 2 and 3 give the right answers the SPI chain works, so the fault is on the CAN side: the wiring to the transceiver, the termination, or the controller. If rung 3 gives 0x00000001 but STATUS bit 0 never goes low after a TX_SEND write, the trigger is not reaching the controller — which points at the register bank, or at your write not having bit 0 set.

c) It should contain what you did, what you observed and what you have already ruled out. For example: *"Rung 2 gives the right masking for TX_ID (0xFFFFFFFF reads back as 0x000007FF), so the SPI chain and the register bank work. STATUS reads as 0x00000001. After a write of 0x1 to TX_SEND, bit 0 never goes low, and nothing shows on tx_bus. We have verified that our command byte is 0x85 and that bit 0 is set in the data."* That is a report somebody can act on; "it does not work" is not.

Exercise 13.4 The termination

page 85

- a) About 60 Ω : two 120-ohm resistors in parallel, one at each end.
- b) There are three resistors instead of two — three 120 Ω in parallel give 40 Ω . Most likely a node in the middle of the bus has its termination switched on.
- c) There is only one resistor; the other is missing or not connected. It shows when the cable gets longer, the speed higher or the environment noisier — that is, typically when the design is moved off the lab bench, which is the worst possible moment.

Chapter 14 — Bus analysis**Exercise 14.1 Read your own traffic**

page 90

The exercise asks for measurements; the questions are answered here.

- a) They should match exactly what you wrote to TX_ID, TX_DLC and the data registers. If they do not, the fault is in the packing, not in the transfer — the analyser decodes what is actually on the wires.
- b) The order is what the analyser adds. A swap of TX_DATA_LO and TX_DATA_HI gives a frame with the right identifier and the right length but the wrong byte order, and this is where it first becomes visible: every test on the laptop may have been green if the test made the same mistake as the code.
- c) Among others SOF, RTR, IDE, r0, the CRC field, the ACK slot and EOF. The register map exposes only the identifier, the DLC and the data — the controller handles the rest, and the analyser is the only place where you see them.

Exercise 14.2 The other way round

page 90

The exercise asks for measurements; the questions are answered here.

- a) Yes, if RX valid was set and you read the registers. If it does not: check that the identifier is one your node receives, and that the bit rate is the same in the analyser.
- b) Bit 1 should go high when the frame is received and low only when you write 0x1 to RX_ACK. If it stays set, the acknowledgement has not gone through — check that bit 0 really is set in the written word.
- c) The second frame overwrites the first if you have not had time to acknowledge it. The controller holds exactly one received frame, and the newest wins; that is documented and deliberate. With a receive() taking around 270 μ s and frames taking 130 μ s, you do not have time.

Exercise 14.3 The acknowledgement

page 90

- a) Yes. The analyser is a real CAN controller and pulls the ACK slot low on every frame it receives with a correct CRC. In the trace it shows as the frame being registered without an acknowledgement error.
- b) The error flag remains *untouched*. This design never reads the ACK slot back, so it cannot detect that nobody acknowledged — the transmission completes exactly as if everything had gone well.
- c) A complete CAN controller would have detected the acknowledgement error, incremented its transmit error counter and **retransmitted the frame**, over and over until somebody answered or the counter reached bus off. A lone node on a real CAN bus therefore transmits the same frame for ever. This controller does not, because it has neither ACK read-back nor automatic retransmission — two of the documented simplifications.

Exercise 14.4 The bottleneck

page 90

- a) It depends on how often your application calls `send()`. With one frame per iteration in a tight loop the load is low by CAN standards — a few per cent — since the SPI link slows you down long before the bus fills up.
- b) A `send()` is six SPI transactions of around $45\ \mu\text{s}$ each, that is, about **$270\ \mu\text{s}$** . A maximal CAN frame is around 130 bits, that is, about **$130\ \mu\text{s}$** at 1 Mbit/s.
- c) **The SPI link.** It takes roughly twice as long as the frame it sends, so the CAN bus stands waiting for you. That matches the calculation in §8.3 — and this is where you can see with your own eyes that it did, which is why the chapter asks you to measure instead of trusting the table.

About the written test

The test is written and is done with pen and paper. The permitted aids are a crib sheet on one A4 sheet, both sides, and a hexadecimal calculator. **The register map is handed out with the test** — the indices do not have to be memorised, but reading the map and knowing what it means is all the more necessary.

Byte sequences, checksum calculations, register packing and SPI transactions have to be doable by hand.

Structure

The test has two parts, of roughly equal size:

Part	Content	Marks
A	Your own protocol: frame design, parsing, addressing, the error model and the reliability chain (chapter 1–chapter 3).	12
B	CAN and the driver: arbitration, the register map, the SPI transaction, the architecture and the limitations (chapter 4–chapter 12).	13

A pass requires marks in *both* parts. An empty part B cannot be compensated for with a perfect part A.

Pass and distinction questions

The pass questions check basic understanding: building and reading a frame, computing a checksum, following a parser through a byte stream, reading the register map, packing a frame into registers, decoding an SPI transaction, and explaining arbitration.

The distinction questions require reasoning: the reliability chain and its consequences, analysis of faults in a byte stream, why sticky status bits are needed, the boundaries of the layered architecture, and what the design’s limitations make impossible for the driver.

The practice test

The next chapter is a complete practice test in the same format and with the same distribution of marks as the real one. The answers are in the chapter after that — with the calculations, not just the results.

Do it with pen and paper, and read the answers only afterwards.

APPENDIX C

Practice test

25 marks in total. **Pass:** at least 12 marks, with at least 4 in each part. **Distinction:** at least 19 marks.

Answer briefly and clearly. Show the calculations where you calculate.

Part A — your own protocol (12 marks)

The checksum is a 16-bit sum of **every byte up to the CHK field**. Fields larger than one byte (SOF, SEQ, CHK) are sent big-endian.

SOF (2) | LEN (1) | TYPE (1) | DST (1) | SRC (1) | SEQ (2) | DATA (N) | CHK (2)

SOF = 0xA5F7. Frame types: Ping = 0x00, Pong = 0x01, StatusRequest = 0x02, StatusResponse = 0x03, Ack = 0x04, Nack = 0x05.

Exercise C.1 Build two frames

4 MARKS

- A node with address 0x17 sends a StatusRequest to a sensor with address 0x25. The sequence number is 0x7F05. Write out the whole frame byte by byte. (2 marks)
- The sensor answers with a StatusResponse containing the temperature 0x16 (one byte). Write out that frame. (2 marks)

Exercise C.2 The frame parser

2 MARKS

- Describe briefly what the parser does in WaitForSof1 and in WaitForSof2. (1 mark)
- The parser is in WaitForSof2 and receives a byte that is not 0xF7. What should it do? There is one answer that is almost right and one that is right — justify which is which. (1 mark)

Exercise C.3 Bus and routing

2 MARKS

- a) Does the routing happen in the bus or in the node? Justify your answer. (1 mark)
- b) A node receives a frame with a correct checksum, but DST is not the node's address. What should the node do, and why should it *not* send a Nack? (1 mark)

Exercise C.4 Reliability

2 MARKS

- a) Node A sends a StatusRequest and gets no Ack. Describe the flow with timeout and retransmission. Should SEQ be the same or a new one, and why? (1 mark)
- b) The Ack frame is lost on its way back, so A retransmits. B receives the same frame a second time. What should B do, and what should B *not* do? (1 mark)

Exercise C.5 Byte stream and error model

2 MARKS

A node receives:

```
32 74 00 FF A5 F7 03 03 25 17 7F 05 10 20 30 02 C2
```

- a) At which index does the frame begin, and what should the parser do with the data before it? (1 mark)
- b) Assume instead that the byte 0x20 is dropped. What happens to the interpretation, and which state does the parser end up in? (1 mark)

Part B — CAN and the driver (13 marks)**Exercise C.6 The CAN bus**

3 MARKS

- a) What is a CAN ID, and which **two** roles does it have at the same time? (1 mark)
- b) Two nodes start transmitting at the same time. Node A has 0x100, node B has 0x0FF. Which one wins, what does the loser do, and why is no bus master needed? (1 mark)
- c) List **three** mechanisms from part A that you did not have to implement in software, and say where in the system they are performed instead. (1 mark)

Exercise C.7 The register map

4 MARKS

- a) You are to send a frame with id = 0x2A7, dlc = 5 and the data bytes 01 02 03 04 05. Write out the five register values to be written, in the right order. Why does the order matter? (2 marks)
- b) You read RX_ID = 0x00000481, RX_DLC = 0x00000003, RX_DATA_L0 = 0xDEADBEE0, RX_DATA_HI = 0x00000000. Which identifier did the frame come from, which data bytes does it contain, and what are frame.data[3] to frame.data[7] to contain after receive()? (1 mark)

- c) Which register has to be written before the next frame can be received, and with which value? What happens if the driver forgets it? (1 mark)

Exercise C.8 The SPI transaction

3 MARKS

The following bytes go out on MOSI. For each one: read or write, which register, and which value? (2 marks)

- a) 81 00 00 07 FF
- b) 85 00 00 00 01
- c) 8B 00 00 00 00
- d) C2 00 00 00 03

- e) Write out the complete five-byte transaction for **reading** RX_DLC. (1 mark)

Exercise C.9 Sticky bits and limitations

3 MARKS

- a) The controller signals with pulses one clock cycle long, 20 ns at 50 MHz. An SPI transaction takes around 40 μ s. Explain why the register bank is needed, and why STATUS bit 0 has to be cleared by a *write* instead of automatically on a read. (1 mark)
- b) Your node transmits a frame and loses the arbitration. STATUS bit 0 comes back exactly as on a successful transmission. What should the driver do to find out how it went, and what can it still *not* find out? (1 mark)
- c) `clearError()` writes 0x0 to `ERROR_FLAGS`, while `receive()` writes 0x1 to `RX_ACK`. A student thinks that is inconsistent and writes 0x1 in both. What is the symptom, and when is it discovered? (1 mark)

Answers to the practice test

The calculations are included, not just the results.

Part A

Exercise C.1 Build two frames

page 121

a) $LEN = 0$ since a `StatusRequest` has no payload, $TYPE = 0x02$, $DST = 0x25$, $SRC = 0x17$, $SEQ = 0x7F05$.

The checksum: $A5 + F7 + 00 + 02 + 25 + 17 + 7F + 05 = 165 + 247 + 0 + 2 + 37 + 23 + 127 + 5 = 606 = 0x025E$.

```
A5 F7 00 02 25 17 7F 05 02 5E
```

b) Now the addresses swap places: $DST = 0x17$, $SRC = 0x25$. $LEN = 1$, $TYPE = 0x03$, and one data byte $0x16$. SEQ is the same.

The checksum: $606 - 0x02 + 0x03 + 0x01 + 0x16 = 606 - 2 + 3 + 1 + 22 = 630 = 0x0276$.

```
A5 F7 01 03 17 25 7F 05 16 02 76
```

Exercise C.2 The frame parser

page 121

a) In `WaitForSof1` the parser discards every byte that is not $0xA5$. It stands still until the stream starts to look like a frame. In `WaitForSof2` it waits for $0xF7$, and moves on to `WaitForLen` if it arrives.

b) The *almost* right answer is to go back to `WaitForSof1` and discard the byte. The *right* one is to stay in `WaitForSof2` if the byte is $0xA5$.

The reason: the byte that just arrived may be the first byte of a real frame. If the parser discards it, it misses a valid frame beginning immediately after a false start of SOF — which happens as soon as $0xA5$ occurs in a payload just before a real frame start.

Exercise C.3 Bus and routing

page 122

a) In the node. The bus is a physical wire delivering every byte to every node; it cannot read addresses, and it knows nothing about frames. Every node therefore has to compare DST with its own address itself.

b) Ignore the frame entirely. It should *not* send a Nack, since the frame is not faulty — it is simply addressed to somebody else. If every node sent a Nack for every frame it was not the receiver of, every message on a bus with n nodes would generate $n - 2$ incorrect negative acknowledgements, and the sender would also be unable to tell them from a real error.

Exercise C.4 Reliability

page 122

a) A starts a timer when the frame is sent. If no Ack arrives before the timer expires, A retransmits the frame and restarts the timer, up to a fixed number of attempts; after that a permanent error is reported.

SEQ is to be the **same**. A new sequence number would make the retransmission a new message, and B would not be able to recognise it as a repeat — which is exactly what duplicate detection needs.

b) B is to **send Ack again**, and **not** run the application logic a second time.

Not running the logic again is the point of duplicate detection: the command has already been carried out. Acknowledging anyway is the part that gets forgotten — A is still waiting for its acknowledgement, and silence only leads to yet another retransmission and thereby to an endless cycle.

Exercise C.5 Byte stream and error model

page 122

a) At index 4 (zero-indexed): 32 74 00 FF is junk, and A5 F7 at index 4–5 is the SOF.

The parser discards the preceding bytes: they are received in `WaitForSof1` and rejected one by one since none of them is 0xA5.

b) LEN is still 3, so the parser collects three data bytes — but they are now 10 30 02 instead of 10 20 30. It then reads C2 as the first checksum byte and waits for the second.

The parser therefore ends up in `WaitForChk2` and stays there until the next byte arrives, whatever it is. When it does, the framing is complete but the checksum does not match, so `extractFrame()` returns `false` and the frame is rejected.

Note that `isFrameReady()` may very well become true — the framing succeeded. It is the validation that catches the error, and that is why the two questions are kept apart.

Part B

Exercise C.6 The CAN bus

page 122

- a) The identifier describes **what the frame contains** — not who it is for. The two roles are:
1. **Content:** the receivers filter on it and decide for themselves whether the frame concerns them.
 2. **Priority:** it decides who wins an arbitration, where a lower numerical value wins.
- b) **B wins**, since 0x0FF is numerically lower than 0x100. In binary, eleven bits: A transmits 00100000000 and B 00011111111, and at bit 3 A transmits recessive where B transmits dominant.

The loser A detects it immediately — it transmits a recessive bit and reads back a dominant one — stops transmitting, becomes a receiver for B's frame, and tries again when the bus is free.

No bus master is needed since the bus is itself an AND function: a dominant bit overwrites a recessive one, so the conflict is resolved by the electronics as it arises. No frame is destroyed and no time is lost.

- c) Three of the following: frame synchronisation (the controller, SOF and EOF), the length field (the controller, the DLC), the checksum (the controller, CRC-15), the acknowledgement (the controller, the ACK bit inside the frame), collision handling (the bus itself, through arbitration).

Exercise C.7 The register map

page 122

- a) Data byte 0 sits in the most significant bits, and TX_DATA_L0 carries the *first* four bytes:

```
TX_ID      = 0x000002A7
TX_DLC     = 0x00000005
TX_DATA_L0 = 0x01020304
TX_DATA_HI = 0x05000000
TX_SEND    = 0x00000001
```

The order matters because the write to TX_SEND does not store a value but *triggers a transmission*. The controller then reads the other registers. If the trigger is written first, the previous frame's contents are sent.

- b) RX_ID = 0x481 gives the identifier **0x481** (within 11 bits, so valid). RX_DLC = 3 means three data bytes, and RX_DATA_L0 = 0xDEADBE00 gives bytes 0–3 as DE AD BE 00. The frame therefore contains **DE AD BE**.

frame.data[3] to frame.data[7] are to be **zero**. The driver is to mask against dlc: the bytes above the received length are not the frame's content, even if the hardware happens to zero them.

- c) RX_ACK, with the value 0x1 — bit 0 has to be set.

If it is forgotten, STATUS bit 1 stays set forever. Every `receive()` then returns `true` with the same frame, in an endless loop, and no new frame can be received.

Exercise C.8 The SPI transaction

page 123

The command byte is `W` in bit 7, zeroes in bits 6–4, and the register index in bits 3–0.

- a) `0x81` = write bit set, index 1: a **write to TX_ID** with the value `0x000007FF`. That is the largest value the register can hold — eleven bits of ones.
- b) `0x85` = write, index 5: a **write to TX_SEND** with the value `0x1`. It **triggers a transmission**; the register stores nothing.
- c) `0x8B` = write, index 11: a **write to ERROR_FLAGS** with the value `0x0`. That is the *only* write that clears the error bit — a non-zero value would have been ignored.
- d) `0xC2` = `1100 0010`. Bit 7 is set (write) *and bit 6 is set*, but bits 6–4 are reserved and have to be zero. The command is therefore **invalid**: no write is committed, no read is latched. The slave consumes all five bytes anyway and returns to idle, and reports nothing — there is no error channel at this layer.
- e) A read has bit 7 low, so the command byte is just the index. `RX_DLC` has index 7, and the four data bytes are dummy bytes:

```
07 00 00 00 00
```

Exercise C.9 Sticky bits and limitations

page 123

- a) At 50 MHz one clock cycle is 20 ns, and 40 μ s corresponds to $40 \cdot 10^{-6} \cdot 50 \cdot 10^6 = 2000$ cycles. The probability that a poll happens to hit the single cycle the pulse lasts is therefore about 1 in 2000: the software would miss nearly every event. The register bank turns the pulse into a **level that stays**, so that it can be polled.

The clearing has to be a write because an automatic clear on a read makes it impossible to read the status twice — and thereby impossible to check a bit first and act on it afterwards. An explicit clear also separates *I have seen the event* from *I have handled it*, and it is the software, not the hardware, that knows when the latter is true.

- b) The driver is to wait until STATUS bit 0 comes back — the port is free — and **then read ERROR_FLAGS**. If the error bit is set, the transmission did not go through.

It still **cannot** find out *why*. The error flag is a single bit for four causes: lost arbitration, stuff error, failed CRC and a received remote frame. Retry logic treating them differently therefore cannot be written against this interface.

- c) `ERROR_FLAGS` is a latch-and-clear-on-zero construction: **only** a write where the whole word is zero clears the latch. A write of `0x1` does nothing at all.

The symptom: the error bit never goes low. `hasError()` returns `true` forever after the first error, and all error handling above stops working — a retry loop reading the error flag will believe that every transmission fails.

It is **not** discovered by a test that only checks which bytes were sent, since the transaction looks correct. It is discovered by a test against a simulated register bank, which models the clearing rule — or, considerably later and more expensively, at bring-up.